


# A SMARTPHONE-CENTRIC, LOW-INFRASTRUCTURE IOT PARKING MANAGEMENT SYSTEM.

*BLE discovery, backend-authorised entry and exit, and a servo gate built from one ESP32*

*Project report submitted in partial fulfillment of the requirements of the course*

**SOFTWARE ENGINEERING PROJECT**

*Submitted by*

**Hari Hara Sudhan**
**24BAI1091**

*Under the guidance of*

**Dr. Elakya R**
**School of Computer Science and Engineering (SCOPE)**

**Fall Semester 2026–27**

# Contents


**1 Introduction** **1**
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives . . . 2
- 1.4 Scope and Limitations . . . 2

**2 Literature Survey and Existing Systems** **4**
- 2.1 Review of Existing Systems . . . 4
  - 2.1.1 Manual ticketing and locally controlled barriers . . . 4
  - 2.1.2 RFID tags and electronic toll collection . . . 4
  - 2.1.3 Camera-based number-plate recognition . . . 4
  - 2.1.4 IoT occupancy sensing and smart-parking surveys . . . 4
  - 2.1.5 BLE beacons and phone-based parking . . . 5
  - 2.1.6 Commercial mobile parking applications . . . 5
- 2.2 Gaps Identified and Proposed Solution . . . 5

**3 Software Process Model and Project Planning** **7**
- 3.1 Choice of Process Model . . . 7
- 3.2 Project Schedule . . . 7
- 3.3 Risk Analysis . . . 8
- 3.4 Effort Estimation . . . 8

**4 Requirements Analysis** **12**
- 4.1 Functional Requirements . . . 12
- 4.2 Non-Functional Requirements . . . 12
- 4.3 Software Requirements Specification Summary . . . 12
- 4.4 Hardware and Software Requirements . . . 12

**5 System Design** **16**
- 5.1 System Architecture . . . 16
- 5.2 Use Cases . . . 16
- 5.3 Layered Structure . . . 17
- 5.4 Session Flow . . . 18
  - 5.4.1 Entry . . . 18
  - 5.4.2 Session state machine . . . 18
  - 5.4.3 Gate decision table . . . 19
  - 5.4.4 BLE interaction design . . . 19
- 5.5 Database Design . . . 20
- 5.6 UI Design . . . 21

5.7 Security Design . . . . . 21

**6 Implementation** **23**
6.1 Technology Stack . . . . . 23
6.2 Module Description . . . . . 23
6.2.1 Identity module . . . . . 23
6.2.2 Vehicle module . . . . . 23
6.2.3 Facility and administration module . . . . . 23
6.2.4 Session module . . . . . 25
6.2.5 Billing and payment module . . . . . 25
6.2.6 Gate authorisation module . . . . . 25
6.2.7 Device and command module . . . . . 25
6.2.8 Scheduled maintenance module . . . . . 25
6.2.9 Mobile application modules . . . . . 25
6.2.10 Firmware modules . . . . . 26
6.3 Key Code Excerpts . . . . . 26
6.3.1 Session creation under a row lock . . . . . 27
6.3.2 Payment freezes the fee . . . . . 27
6.3.3 Exit verdict with a grace re-open . . . . . 28
6.3.4 BLE round trip in the application . . . . . 28
6.3.5 Firmware transport selection . . . . . 29

**7 Software Tools Used** **30**
7.1 Git and GitHub . . . . . 30
7.2 Docker and Docker Compose . . . . . 30
7.3 Postman . . . . . 31
7.4 Visual Studio Code . . . . . 31
7.5 Arduino CLI and Arduino IDE . . . . . 33
7.6 Summary . . . . . 33

**8 Testing** **35**
8.1 Test Plan and Strategy . . . . . 35
8.2 Test Cases . . . . . 36
8.3 Test Results and Bug Log . . . . . 36

**9 Results and Discussion** **40**
9.1 Output of the Working System . . . . . 40
9.1.1 End-to-end walkthrough . . . . . 40
9.2 Analysis Against Objectives . . . . . 46

**10 Conclusion and Future Enhancements** **48**
10.1 Conclusion . . . . . 48
10.2 Future Enhancements . . . . . 48

# 1. Introduction

## 1.1 Background and Motivation

Parking is one of the last everyday transactions that still expects the driver to interact with dedicated equipment. A typical urban facility installs a barrier, a ticket dispenser or an RFID reader, a payment kiosk and, increasingly, a set of cameras for number-plate recognition. Each item adds capital cost, a maintenance visit and an independent attack surface, and all of it must be commissioned before the first vehicle can enter. The hardware exists because a car has no identity that a server can verify. The driver, however, does: a smartphone can authenticate a person to a backend, display a price and authorise a payment.

Bluetooth Low Energy (BLE) completes the picture at the gate. BLE advertisements are cheap to broadcast, readable by any modern phone, and can carry a facility identifier without a network connection. Research on BLE-beacon parking has shown that proximity-based discovery and automated payment are practical on commodity phones [1], and surveys of IoT parking report the same direction: sensing and connectivity move into low-cost devices while the business logic moves into a backend [2, 3]. What most of that work optimises is *occupancy sensing* — finding a free bay — while the barrier itself remains a conventional, locally controlled device.

DWARA takes the opposite emphasis. It is not an occupancy system; it is a gate controller. One ESP32 advertises its facility over BLE. The React Native application in the driver's phone discovers the gate, starts a parking session on the backend, and then writes the session identifier to the gate. The gate asks the backend whether that session may pass, and moves a servo only when the answer is `open`. There is no camera, no RFID tag, no ticket and no vehicle sensor. The phone replaces the ticket, and the backend replaces the local decision.

The Indian toll ecosystem shows both the promise and the limit of vehicle-bound tags: the National Electronic Toll Collection (NETC) programme issues a FASTag RFID sticker per vehicle and settles tolls through a central system [4]. That model works because a toll is a fixed charge at a fixed point. Parking is neither: the charge depends on how long the vehicle stayed, and the exit decision depends on whether it has been paid. DWARA therefore attaches identity to the *person* and time to the *server*, which is what makes per-second billing possible.

## 1.2 Problem Statement

Design and build a parking gate system in which:

1. a driver discovers a nearby facility without scanning a QR code, typing a code or sharing a location;

2. every entry and exit is authorised by a server, so that a forged client cannot manufacture permission to open a gate;

3. billing is proportional to the actual time parked and is frozen the moment payment succeeds, with no clock controlled by the phone or the gate;
4. the parking state machine cannot be driven into an invalid configuration by replaying requests, double-tapping a button or abandoning a session;
5. the complete system — backend, mobile application and gate firmware — can be built, run and verified on a developer machine before any hardware is flashed.

## 1.3 Objectives

1. **Frictionless discovery.** Advertise the facility identity from the gate itself over BLE and resolve facility details from the backend.
2. **Server-authoritative gate control.** Make the firmware a courier that acts only on a backend verdict and fails closed when the backend is unreachable.
3. **Auditable per-second billing.** Compute every amount from the backend clock and the facility's pricing policy, and never accept a time or a price from the client.
4. **A safe session lifecycle.** Enforce one active session per user and per vehicle, abort sessions that never reach a gate, and define a verdict for every illegal transition in advance.
5. **A complete three-surface deliverable.** Deliver the Spring Boot backend, the Expo/React Native application and the ESP32 Arduino firmware from one repository with one configuration model.
6. **Reproducible verification.** Provide one command that builds the images, boots the real application, runs the automated suite, enforces a coverage gate and publishes reports.

## 1.4 Scope and Limitations

The version-one scope is deliberately narrow. Table 1.1 separates what is implemented from what is excluded, and lists the limitations that follow from the design decisions rather than from missing work.

Four limitations are accepted by design and are not defects:

- **Plaintext BLE.** The link carries a session identifier, not a credential, and the gate revalidates every request with the backend, so the worst outcome of writing a foreign identifier is a closed gate.
- **Mock payment.** The provider sits behind a two-method interface and always succeeds; no money moves. Replacing it is an implementation of the interface, not a redesign.
- **No sensors.** The gate closes on a fixed eight-second timer. It cannot detect a tailgating vehicle, and it cannot detect a jammed arm except by a bounded settle window. A mechanical push-through override is required hardware.
- **Foreground BLE.** Discovery and gate commands need the application in the foreground and the phone within a few metres of the gate.

| In scope | Out of scope (version one) |
|---|---|
| Registration, login and profile restore with JWT | Administrator user role or an operator web console |
| Vehicle CRUD with number plate and class (MC/LMV/HMV) | Real payment-gateway integration, refunds and wallets |
| BLE facility discovery from the gate's advertisement | GPS, QR-code or manual-code discovery |
| Backend-authorised entry and exit with reason codes | Multi-gate facilities or more than one servo per site |
| Per-second billing with frozen fees and receipts | Minimum fees, grace periods, daily caps, occupancy tracking |
| Session history with status filter and paging | Heartbeats, over-the-air firmware updates, remote configuration |
| Device provisioning and remote open/keep-closed commands | BLE pairing or encryption, iOS background BLE |
| Dockerised build, test workflow and coverage gate | Production TLS termination and certificate management |

Table 1.1: Scope of the delivered system.

# 2. Literature Survey and Existing Systems

## 2.1 Review of Existing Systems

Parking technology has evolved along four largely independent lines: mechanical ticketing, vehicle-bound electronic tags, camera-based identification, and IoT occupancy sensing. Each solves a real problem, and each leaves a specific gap that shapes the design of DWARA.

### 2.1.1 Manual ticketing and locally controlled barriers

The conventional facility issues a ticket at entry, reads it at exit and charges at a kiosk. The ticket is anonymous, so the system cannot associate a stay with a person or a vehicle; revenue protection depends on the barrier and on staff. The controller decides locally, which means a faulty loop detector, a lost ticket or a power interruption becomes an operational incident rather than a query against a record. This model is well understood and cheap per site, but it cannot support account-based billing, history or remote operation without additional hardware.

### 2.1.2 RFID tags and electronic toll collection

Vehicle-bound RFID is the dominant model for tolling. India's National Electronic Toll Collection programme issues a FASTag sticker that a fixed reader activates as the vehicle approaches; the tag identifier is mapped to a prepaid or linked account and the toll is deducted centrally [4]. The strengths are speed and interoperability: the tag is passive, the reader is overhead, and the settlement is centralised. The limitation for parking is structural. The tag identifies a *vehicle*, not a session, and the charge is a fixed toll for a crossing. A parking fee depends on duration and on payment state, so an RFID-only design still needs a separate session and billing system behind it. It also requires a reader at every lane and a tag in every vehicle.

### 2.1.3 Camera-based number-plate recognition

ANPR systems photograph the plate, extract the characters and match them against a record. They remove the tag but introduce a heavy dependency: illumination, weather, plate formats and camera placement all affect accuracy, and each lane needs a camera plus compute. Privacy is a further concern, because plate images are personal data. The approach answers "which vehicle is this?" but not "is this driver allowed to leave, and has the fee been paid?" — those questions still need an application layer.

### 2.1.4 IoT occupancy sensing and smart-parking surveys

A large body of work places a sensor at every bay — ultrasonic, magnetometer, infrared or camera — and reports occupancy to a server. Recent surveys catalogue these architectures,

communication standards and business models [2, 3]. The value is guidance to a free space and utilisation analytics. The cost is one device per bay, which dominates deployment economics for a small facility, and the sensing problem is orthogonal to the access-control problem: knowing that a bay is free does not tell a barrier whether to open.

### 2.1.5 BLE beacons and phone-based parking

BLE has been used mainly for proximity. Mackey et al. pair each parking spot with a BLE beacon and use particle filtering on received signal strength to decide which spot a phone has entered, with an automated payment scheme built on top [1]. The result demonstrates that commodity phones and low-cost beacons are sufficient for reliable proximity decisions. It also concentrates the intelligence on the phone and the cloud rather than on the barrier, and it assumes one beacon per spot.

### 2.1.6 Commercial mobile parking applications

App-based parking payment (for example, phone applications that start and stop a parking stay) is now common. These systems digitise the payment record but usually leave enforcement and the barrier with the operator; the phone proves payment, and a human or a camera checks it. They validate the central idea of DWARA — that a phone account is a usable parking credential — while leaving the gate itself unchanged.

| **Approach** | **Credential** | **Hardware per site** | **Billing basis** | **Main gap** |
|---|---|---|---|---|
| Ticket + local barrier | Paper ticket | Barrier, dispenser, kiosk | Fixed tariff per stay | Anonymous; no account or history |
| RFID / FASTag | Vehicle tag | Reader per lane, tags per vehicle | Fixed charge per crossing | Vehicle-bound; no duration model |
| ANPR cameras | Number plate | Camera + compute per lane | Whatever the backend adds | Cost, accuracy, privacy |
| Bay sensors | None | One sensor per bay | Time (computed centrally) | Does not control access; cost per bay |
| BLE beacons (research) | Phone app | One beacon per spot | Usage time | Assumes per-spot beacons |
| Mobile payment apps | Phone account | None at the gate | Time | Barrier and enforcement stay manual |
| **DWARA** | **Phone account + JWT** | **One ESP32 + one servo** | **Per second, backend clock** | **Plaintext BLE; single gate (v1)** |

Table 2.1: Comparison of existing approaches with the proposed system.

## 2.2 Gaps Identified and Proposed Solution

Table 2.1 makes the gaps concrete. Five matter for this project.

1. **The barrier is trusted too much.** In ticketing, RFID and camera systems the local controller decides whether to open. DWARA moves the decision to the backend and reduces the controller to a courier with a device credential. A compromised phone can annoy the backend; it cannot open a gate.

2. **Identity is attached to the wrong object.** Tags and plates identify vehicles, and tickets identify nothing. DWARA attaches identity to the account behind the phone, and then binds that account to a vehicle at session creation.

3. **Duration is an afterthought.** Toll-style charging is per crossing, and bay sensors compute time but do not gate access. DWARA treats the duration as the primary quantity: the clock starts when the backend authorises entry and stops when payment succeeds.

4. **Per-bay hardware does not scale down.** A small facility cannot justify a sensor at every bay. DWARA needs one ESP32 and one servo for the whole site and no per-bay device.

5. **Verification is rarely reproducible.** Student and prototype projects often demonstrate a flow manually. DWARA ships a containerised verification workflow with a coverage gate and published reports, so the evidence can be regenerated by anyone with Docker.

The proposed system therefore combines three ideas that exist separately in the literature: BLE proximity discovery from research beacons [1], account-based settlement from tolling [4], and server-side session management from IoT parking platforms [2]. Its contribution is the arrangement: a phone-carried session identifier, a single gate that re-validates every action, and a lifecycle that makes duration and payment the only inputs to the exit decision.

# 3. Software Process Model and Project Planning

## 3.1 Choice of Process Model

The project followed an **iterative, sprint-based (Agile) model** rather than a single waterfall pass. Four properties of the problem made the choice almost forced.

1. **The hardware was the least predictable part.** Servo power, BLE behaviour on iOS and Wi-Fi reliability could only be characterised by building and running the gate. A plan that fixed every detail up front would have been rewritten after the first bench test.
2. **Correctness was cheap to check but expensive to guess.** The session lifecycle has a small number of states and many illegal transitions. Writing the transitions down as tests and then iterating until they passed was faster and safer than elaborating them on paper.
3. **The work could be sliced vertically.** Every increment could deliver a working path through all three surfaces — for example, "start a session and see it on the dashboard" — instead of completing one layer at a time. Vertical slices kept the system demonstrable after every sprint.
4. **The team was small and part-time.** Short sprints with a single demonstrable outcome tolerate academic schedules far better than a monolithic phase plan.

The model was Scrum-inspired but deliberately lightweight: a single prioritised backlog derived from the architecture specification, one-week to ten-day sprints, a working demonstration at the end of each sprint, and a written decision log when a question could not be settled by the specification. There were no daily stand-ups and no story-point estimates; progress was measured by what could be executed.

The definition of done for every increment was the same and is worth stating because it explains the shape of the repository: *the feature works end to end, its service-layer lines are covered by tests, the relevant document is updated, and* `mvn verify` *is green*. Documentation was treated as part of the increment, not as a final phase, which is why the API reference, BLE protocol and firmware notes match the code.

Waterfall was rejected because the hardware findings would have arrived in the verification phase, after the backend design had been frozen. A pure prototype model was rejected because the deliverable is an assessed project with a written specification, a documented threat model and reproducible evidence; those artefacts need a plan that produces them incrementally.

## 3.2 Project Schedule

Development ran for eleven weeks, with a short baseline week at the start and a final verification week. Sprints were sequenced by dependency: identity first, because every other endpoint

needs a caller; money second, because the exit gate depends on payment; hardware third, once the contract was stable; verification and writing last, against a system that no longer changed shape.

| Sprint | Dates | Goal and deliverable |
|---|---|---|
| S0 | 07–11 Aug | Baseline and decisions. Repository, architecture specification, decision log D1–D23, risk list. |
| S1 | 12–18 Aug | Identity. Registration, login, JWT issue and verification, security filter chain. |
| S2 | 19–25 Aug | Vehicles and facilities. Owner-scoped CRUD, facility lookup by code, admin-key surface, demo seeder. |
| S3 | 26 Aug–01 Sep | Sessions and money. Creation, anti-abuse rule, per-second fee, mock payment, cancellation, state machine. |
| S4 | 02–08 Sep | Gate authorisation and devices. Entry/exit verdicts, device provisioning and secrets, remote-command queue. |
| S5 | 09–15 Sep | Lifecycle hardening. Stale-session sweep, grace window, exception advice, history paging. |
| S6 | 16–22 Sep | Mobile application. Typed REST client, token persistence, eleven screens on real data. |
| S7 | 23–29 Sep | BLE. Discovery, GATT round trip, real transport in the application; firmware transports and servo. |
| S8 | 30 Sep–08 Oct | Firmware completion. Remote-command poller, stall watchdog, serial bridge for the demo. |
| S9 | 09–15 Oct | Verification. Docker Compose test workflow, coverage gate, report collection. |
| S10 | 16–22 Oct | Documentation and hand-over. API reference, protocol and hardware guides, Postman collection, this report. |

Table 3.1: Sprint plan.

Sprints S1–S5 built the backend and its tests; S6–S8 built the phone and the gate against that contract; S9 built the machinery that proves the whole thing works. The ordering meant that the firmware was written against an API that had already been tested, so a gate failure could be attributed to hardware or transport rather than to an unfinished backend.

## 3.3 Risk Analysis

Risks were reviewed at the start of each sprint. Table 3.2 lists the significant ones, the mitigation chosen and what actually happened.

## 3.4 Effort Estimation

A simple decomposition estimate was used: each module was sized by the number of distinct behaviours it had to support, and reviewed at the end of the sprint in which it was built. Table 3.3 records the planned effort and the measured size of the delivered code. The size figures are counted from the repository; the effort figures are planning estimates for a three-person team working part-time.

The distribution is deliberately backend-heavy. The mobile application and the firmware are thin by design: neither holds a decision that the backend could make. The largest single

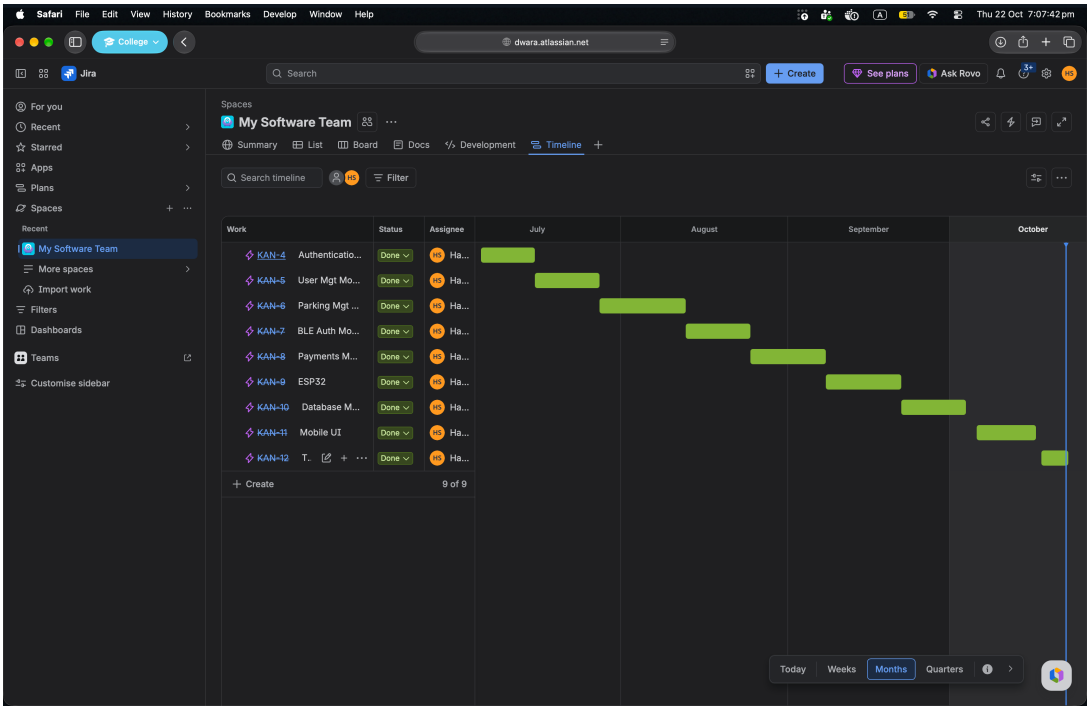



Figure 3.1: Gantt chart of the ten-week schedule, showing the dependency between identity, billing, hardware and verification work.

risk-reduction item was not a feature but the test workflow, because it turned every later change into a verifiable one.

| Risk | Likelihood | Impact | Mitigation and outcome |
|---|---|---|---|
| Servo browns out the ESP32 when the gate moves | High | High | External 5 V supply, common ground and a decoupling capacitor were made mandatory; the actuator is behind an interface so an LED stand-in can be swapped in one line. The servo build is current. |
| BLE notification lost because the phone subscribed after the write | Medium | High | The client subscribes to the status characteristic before writing and filters notifications by session id. Implemented. |
| Backend and app disagree about the response shape | High | Medium | One envelope for every application endpoint, two documented exceptions for firmware, and a typed client that rejects unknown shapes. |
| Double payment through a double tap or retry | Medium | High | Session row locked during payment; the second transaction sees `PAYMENT_PENDING` and is refused. |
| Concurrent session creation creates two sessions | Low | High | The user row is locked for the transaction, serialising the read-then-write check. |
| Coverage gate blocks iteration | Medium | Medium | Tests are written with the feature, not afterwards; the gate is scoped to the service layer so generated code does not dilute it. |
| Hardware unavailable for testing | High | Medium | The serial-bridge transport allows the full backend flow without Wi-Fi; BLE and servo behaviour remain documented as bench steps. |
| Secrets committed to the repository | Medium | High | `.env` is ignored, sample values live in `.env.example`, device secrets are hashed, and the admin key has no default. |
| Scope creep from payment or occupancy features | Medium | Medium | Both were written into the non-goals and revisited only as future work. |
| Test environment differs from deployment | Low | High | No H2 and no Testcontainers: tests run against PostgreSQL 16, the same image the stack uses. |

Table 3.2: Risk register with the mitigation that was adopted.

| Work package | Planned person-days | Size (lines) | Notes |
|---|---|---|---|
| Requirements and design | 6 | — | specification, decision log, threat model |
| Backend domain and services | 14 | 3 275 | Java, entities to controllers |
| Backend tests | 8 | 2 240 | 15 classes, 118 tests, Mockito |
| Mobile application | 14 | 2 278 | TypeScript, 11 screens, REST and BLE |
| Gate firmware | 10 | 1 761 | Arduino, transports, actuator, poller |
| Build and test workflow | 6 | 261 | Compose profile and two shell scripts |
| Documentation and report | 8 | — | six guides, API reference, this report |
| **Total** | **66** | **9 815** | |

Table 3.3: Indicative effort distribution. Size counts exclude blank lines and generated build output; scripts are shell, firmware is C/C++.

# 4. Requirements Analysis

## 4.1 Functional Requirements

Requirements are grouped by module and numbered for traceability. The final column names the evidence that the requirement is met; "test" refers to the automated suite described in Chapter 8, and "walkthrough" to the end-to-end demonstration in Section 9.1.1.

## 4.2 Non-Functional Requirements

## 4.3 Software Requirements Specification Summary

The full specification is maintained in the repository (`dwara-architecture-spec.md`); Table 4.3 is the summary that binds this report.

### Requirement prioritisation

All requirements in Table 4.1 were delivered. Had the schedule slipped, the release order would have been: identity (FR1–FR3), session creation and the anti-abuse rule (FR7–FR8), gate verdicts and payment (FR10, FR15–FR16), then history and remote commands. The payment flow and the exit verdict are inseparable, because a gate that cannot check payment cannot be shipped; history and administration are the only genuinely optional surfaces.

## 4.4 Hardware and Software Requirements

| ID | Requirement | Verified by |
|---|---|---|
| *Identity* | | |
| FR1 | Register with name, e-mail and password. The password is stored only as a BCrypt hash; the role is forced to `USER`. | test |
| FR2 | Log in with e-mail and password and receive a JWT that identifies the account. | test, Postman |
| FR3 | Exchange a stored JWT for the caller's profile so a relaunch restores the session. | test, walkthrough |
| *Vehicles and facilities* | | |
| FR4 | Add, list, update and delete own vehicles, each with a number plate and a class (`MC`, `LMV`, `HMV`). | test |
| FR5 | Look up a facility by its code, including per-second rates for all three vehicle classes. | test |
| FR6 | Create facilities, change pricing and change status through an administrator key. | test, walkthrough |
| *Sessions and billing* | | |
| FR7 | Start a session for one of the caller's vehicles at an active facility; the session begins in `CREATED` with no timestamps. | test |
| FR8 | Refuse a new session when the user already holds one, or when the vehicle already holds one. | test |
| FR9 | Read the live fee of a parked session, computed from the backend clock. | test |
| FR10 | Pay for a parked session; the amount is frozen and the session moves to `PAYMENT_PENDING`. | test, walkthrough |
| FR11 | Cancel a session that has not entered the gate. | test |
| FR12 | Abort `CREATED` sessions that are never validated at a gate. | test |
| FR13 | Page through own session history, newest first, optionally filtered by status. | test |
| *Gate and devices* | | |
| FR14 | Register gate hardware; return the device secret exactly once and store only its hash. | test |
| FR15 | Answer an entry verdict: open only for a `CREATED` session at the same facility, and stamp the entry time. | test |
| FR16 | Answer an exit verdict: open only for a paid session, and allow one grace re-open after exit. | test |
| FR17 | Report every verdict with a machine-readable reason, and deny with a reason rather than an error when the session is unknown. | test |
| FR18 | Queue `OPEN` or `KEEP_CLOSED` to a gate; the next poll consumes the command exactly once. | test |
| *Application* | | |
| FR19 | Discover a gate by BLE advertisement, list nearby facilities and fetch their details. | walkthrough |
| FR20 | Send entry and exit commands over BLE with the session identifier and display the gate's answer, including denial reasons. | walkthrough |

Table 4.1: Functional requirements and their verification evidence.

| Attribute | Requirement and how it is satisfied |
|---|---|
| Security | Three credentials with separate scopes: JWT for users, a static header for administration, a per-device bearer secret for hardware. Passwords and device secrets are stored as hashes; the gate decision is always server-side. |
| Safety | The gate fails closed. An unreachable backend, a malformed answer or an actuator fault never opens the gate; a latched fault requires a reset. |
| Consistency | State changes are transactional. Session creation is serialised on the user row, payment on the session row. |
| Reliability | Health-checked dependencies in Compose; the test runner waits for PostgreSQL and the backend before starting. Stale sessions are reaped so a crash cannot block an account indefinitely. |
| Maintainability | Layered packages (controller, service, repository), DTOs at the boundary, constructor injection, no business logic in controllers. |
| Testability | Every replaceable dependency is an interface (`PaymentService`, `GateControllerClient`, `AuthService`, `GateActuator`). 118 automated tests and a coverage gate. |
| Portability | Backend and tests run in containers; firmware is one Arduino sketch with a single configuration header. |
| Usability | No facility code or location entry: the gate is chosen from a scan. Refusals are shown as the backend's own message. |
| Observability | Public health endpoint, structured gate log lines, JUnit XML and JaCoCo reports produced by every run. |
| Performance | Gate validation is a database read plus a state write; the application allows 15 s for a full BLE round trip including the servo sweep and the 8 s hold. |

Table 4.2: Non-functional requirements.

| Item | Specification |
|---|---|
| Purpose | Phone-controlled parking gate with backend-authorised entry and exit and per-second billing. |
| Product scope | Three deliverables in one repository: Spring Boot backend, Expo/React Native application, ESP32 Arduino firmware. |
| Actors | Driver, attendant/operator, gate controller, scheduled sweep. |
| External interfaces | REST/JSON over HTTP for the application and the administrator; REST/JSON over Wi-Fi for the gate; BLE GATT for the phone; JDBC to PostgreSQL. |
| Data | Users, vehicles, facilities with JSON pricing, sessions, payments, gate controllers and commands. All keys are UUIDs; all money is decimal. |
| Assumptions | One gate per facility; the driver's phone is BLE-capable and runs the application in the foreground; the facility has Wi-Fi for the gate. |
| Constraints | Backend clock is the only billing clock; fixed-timer gate close; plaintext BLE in version one; mock payment provider. |
| Acceptance | Every functional requirement passes its listed evidence; the automated suite is green with a 100% service-layer line-coverage gate; the mobile build type-checks; the firmware compiles for ESP32. |
| Out of scope | Real payments, refunds, occupancy sensing, multiple gates, roles beyond `USER`, OTA updates, background BLE. |

Table 4.3: Software requirements specification summary.

| Category | Requirement |
|---|---|
| Development host | Docker Engine 24 or later with Compose v2 and `bash`. No host JDK or Maven is required for the container path (JDK 21 and the Maven wrapper are needed only for host-side builds). |
| Backend runtime | Java 21, Spring Boot 4.1, Spring Security, Spring Data JPA, JJWT, Actuator; PostgreSQL 16. |
| Mobile toolchain | Node.js with npm, Expo 57, React Native 0.86, TypeScript, `react-native-ble-plx`; a development build on a physical phone for BLE. |
| Firmware toolchain | `arduino-cli` with the ESP32 core, ArduinoJson 7 and ESP32Servo; Huge APP partition scheme for the Wi-Fi and BLE build. |
| Demo tooling | Python 3 with a virtual environment for the serial bridge; Postman or Newman for the API contract collection. |
| Network | Phone and backend on the same LAN, backend port reachable; the gate needs Wi-Fi for production validation. |

Table 4.4: Software requirements.

| Part | Quantity | Purpose / note |
|---|---|---|
| ESP32 20-pin development board | 1 | BLE peripheral and Wi-Fi station; 3.3 V logic. |
| MG90S micro servo | 1 | Gate arm actuator; metal gears, 4.8–6 V. |
| External 5 V supply, at least 1 A | 1 | Servo power; never the ESP32 3.3 V rail. |
| Electrolytic capacitor, 470–1000 $\mu$F | 1 | Across the servo supply, close to the servo, to absorb stall transients. |
| Gate arm and hinge | 1 | Light arm; push-through override recommended. |
| Jumper wires, breadboard, USB cable | — | Wiring, flashing and serial monitor. |

Table 4.5: Gate hardware bill of materials.

# 5. System Design

## 5.1 System Architecture

DWARA is a three-surface system with one source of truth. The phone is an interface, the ESP32 is an actuator with a radio, and the backend owns identity, parking state and money.

Three properties of this shape explain most of the code:

1. **The application never authorises a gate.** It carries a session identifier over BLE and displays the answer; it holds no credential a gate would accept.
2. **The gate never decides.** It holds a device secret that lets it ask, and a servo it may drive only after the answer arrives. Losing Wi-Fi closes gates rather than opening them.
3. **The backend never touches hardware.** It returns a verdict and records the state change, which keeps the mechanism replaceable.

| Component | Responsibility |
|---|---|
| Mobile application | Registration, login and token persistence; vehicle management; BLE discovery; session start; live fee; payment; gate commands; history. |
| Backend | Identity and JWT, vehicles, facilities and pricing, session lifecycle, fee computation, mock payment, gate verdicts, device registry, remote command queue, stale-session sweep, health endpoint. |
| PostgreSQL | Durable state for users, vehicles, facilities, sessions, payments, gates and commands; pricing stored as `jsonb`. |
| Gate firmware | BLE peripheral and GATT service, Wi-Fi client, servo actuation, fixed-timer close, stall watchdog, remote-command poller. |
| Serial bridge | Demonstration-only relay from the firmware's framed serial requests to the backend. |
| Test workflow | Compose profile that boots dependencies, proves the production image is healthy, runs the suite and exports reports. |

Table 5.1: Component responsibilities.

Deployment is a single Compose project. The development stack publishes two services; the test stack adds a third behind a profile, so ordinary development is unaffected.

## 5.2 Use Cases

The two use cases that differ most from a conventional system are UC5 and UC10. In UC5 the driver's tap is a request, not an authorisation: the gate validates it with the backend. In UC10 the operator never contacts the gate; the command is queued and consumed by the gate's next poll.

| Decision | Rationale and consequence |
|---|---|
| BLE advertisement as the facility identifier | No GPS, QR or manual entry; the gate broadcasts `DWARA-<facilityCode>`. |
| One gate serves entry and exit | The direction is a software state, not a second piece of hardware; the facility needs one servo. |
| Backend authorises, firmware obeys | A forged client cannot open a gate; the firmware has no rule engine at all. |
| Backend clock is the only clock | Neither the phone nor the ESP32 can move a bill; all timestamps are server-side. |
| Per-second flat tariff | Trivial to explain and audit; stored per facility as JSON so prices change without a migration. |
| One active session per user and per vehicle | Removes a class of abuse and simplifies the dashboard to a single active session. |
| Payment freezes the fee | The exit gate asks about payment, not about elapsed time, so a queue at the exit cannot increase a bill. |
| Fixed-timer gate close | No sensors; eight seconds is long enough for a car and short enough to limit tailgating. |
| Abstract actuator and transport | Servo, LED, Wi-Fi and serial are interchangeable behind interfaces, which is what makes bench testing possible. |
| Stale-session sweep | A session abandoned before the gate would otherwise block the account forever. |

Table 5.2: Key design decisions.

| ID | Use case | Actor |
|---|---|---|
| UC1 | Register and sign in | Driver |
| UC2 | Manage vehicles | Driver |
| UC3 | Discover a facility over BLE | Driver |
| UC4 | Start a parking session | Driver |
| UC5 | Open the entry gate | Driver, gate |
| UC6 | View the running fee | Driver |
| UC7 | Pay and open the exit gate | Driver, gate |
| UC8 | Review session history | Driver |
| UC9 | Provision a facility and a gate | Operator |
| UC10 | Override a gate remotely | Operator, gate |

Table 5.3: Use-case inventory.

### Detailed use case: open the exit gate (UC7)

## 5.3 Layered Structure

The backend is layered rather than organised as a single inheritance tree: controllers depend on services, services on repositories and domain objects, and every external capability is an interface.

| Field | Description |
|---|---|
| Precondition | The caller owns a session in `PAYMENT_PENDING` with a successful payment; the phone is in the foreground near the gate. |
| Main flow | 1. Driver taps *Open exit gate*. 2. Application writes {`sessionId, intent:"EXIT"`} to the gate. 3. Gate calls `POST /gate/validate` with its device secret. 4. Backend verifies the device, facility, state and payment, stamps the exit and returns `open`. 5. Gate sweeps the servo, reports `OPENED`, holds eight seconds, closes. 6. Application refreshes the session and shows completion. |
| Alternate flow | If the session is unpaid the backend returns `keep_closed/PAYMENT_REQUIRED`; the gate reports `DENIED` and the screen shows the payment prompt. |
| Alternate flow | If the gate already exited within the grace window, the backend re-opens without changing state. |
| Failure | Backend unreachable: the gate reports `DENIED/BACKEND_UNREACHABLE` and stays closed. |
| Postcondition | Session is `EXITED` (or unchanged for a grace re-open); the fee is frozen. |

Table 5.4: Use-case description for the exit flow.

| Package | Contents |
|---|---|
| `controllers` | Six REST controllers (`Auth`, `Vehicle`, `Facility`, `Session`, `Gate`, `Device`) and one exception advice. |
| `services` | Eleven service classes: identity, vehicles, facilities, sessions, billing, gate authorisation, provisioning, device authentication, commands, polling client, scheduler. Four interfaces. |
| `repositories` | Seven Spring Data interfaces, including a locked user lookup, a locked session lookup and a paged history query. |
| `entities` | Seven entities, eleven enums and the `PricingPolicy` record. |
| `dtos` | Request records, response DTOs and the generic `ResponseObject` envelope. |
| `configs` | Filter chain, JWT filter, admin-key filter and session timing properties. |

Table 5.5: Layer contents.

## 5.4 Session Flow

### 5.4.1 Entry

The entry path has three decisions that are easy to get wrong. The session is created *before* the driver reaches the gate, so a failure at the gate does not lose the vehicle; the gate validates the device's facility before reading the session; and the entry timestamp is written inside the same transaction that decides `open`.

### 5.4.2 Session state machine

The session is the central object; every other rule refers to its state.

| From | To | Trigger and rule |
|---|---|---|
| (new) | CREATED | start session, active facility, no active session for the user or the vehicle |
| CREATED | PARKED | entry verdict open; entry time stamped |
| CREATED | ABORTED | explicit cancel, or the sweep after fifteen minutes |
| PARKED | PAYMENT_PENDING | payment succeeds; amount frozen |
| PAYMENT_PENDING | EXITED | exit verdict open with a successful payment |
| EXITED | (re-open) | exit verdict inside the grace window; no state change |

Table 5.6: Legal session transitions. Everything else is refused with a reason.

| Intent | State | Verdict |
|---|---|---|
| ENTRY | CREATED at the same facility | open, session becomes PARKED |
| ENTRY | PARKED, PAYMENT_PENDING, EXITED | keep_closed / WRONG_STATE_FOR_ENTRY |
| ENTRY | ABORTED | keep_closed / STALE_SESSION |
| EXIT | PAYMENT_PENDING with a successful payment | open, session becomes EXITED |
| EXIT | PAYMENT_PENDING without payment | keep_closed / PAYMENT_REQUIRED |
| EXIT | CREATED or PARKED | keep_closed / PAYMENT_REQUIRED |
| EXIT | EXITED inside the grace window | open, no state change |
| EXIT | EXITED outside the grace window | keep_closed / STALE_SESSION |
| any | unknown session | keep_closed / UNKNOWN_SESSION |
| any | device at another facility | keep_closed / WRONG_FACILITY |

Table 5.7: Decision table implemented by the gate service.

### 5.4.3 Gate decision table

### 5.4.4 BLE interaction design

The BLE link is a request channel: it carries a session identifier and an intent, and returns the gate's answer. It carries no credential and no verdict the gate would trust on its own. The service and characteristic layout follows the Generic Attribute Profile of the Bluetooth Core Specification [6].

| Element | Design |
|---|---|
| Advertisement | local name DWARA-<facilityCode>; service UUID 4fafc201-1fb5-459e-8fcc-c5c9c331914b |
| Command characteristic | beb5483e-36e1-4688-b7f5-ea07361b26a8, write |
| Status characteristic | c7d4a2d0-5f3a-4c3e-9b1a-8e2f6d4c5b7a, read and notify |
| Command payload | {"sessionId":"<uuid>","intent":"ENTRY"\|"EXIT"\|"CLOSE"}, at most 128 bytes |
| Status payload | {"sessionId":"<uuid>","result":"OPENED","reason":"NONE"} |
| Timing | 800 ms servo settle, 8 s open hold, 3 s stall watchdog, 5 s command poll, 15 s application round-trip budget |

Table 5.8: BLE protocol design.

## 5.5 Database Design

Seven entities describe the domain. Primary keys are server-generated UUIDs, money is decimal, and enumerations are stored as strings so a database inspection is readable without a lookup table.

| Entity | Fields and role |
|---|---|
| User | userId, unique email, name, passwordHash, userRole, createdAt. Table name users, because user is reserved in PostgreSQL. |
| Vehicle | vehicleId, numberPlate, vehicleType, owning user. |
| ParkingFacility | facilityId, unique facilityCode, name, address, status, PricingPolicy as jsonb. |
| ParkingSession | sessionId, facility, user, vehicle, sessionStatus, createdAt, entryTimestamp, exitTimestamp, exitGateOpenedAt, totalFee. |
| Payment | paymentId, session, amount, status, transactionReference. |
| GateController | controllerId, facility, gateType, firmwareVersion, write-only deviceSecretHash. |
| DeviceCommand | commandId, device, command, status, createdAt, deliveredAt. |

Table 5.9: Entity design.

| Enumeration | Values and meaning |
|---|---|
| SessionStatus | CREATED, PARKED, PAYMENT_PENDING, EXITED, ABORTED; the first three form the active set. |
| PaymentStatus | SUCCESS, with room for failure states. |
| VehicleType | MC, LMV, HMV. |
| GateType | ENTRY, EXIT. |
| GateIntent | ENTRY, EXIT. |
| GateDecision | OPEN, KEEP_CLOSED. |
| DenyReason | NONE, STALE_SESSION, WRONG_STATE_FOR_ENTRY, PAYMENT_REQUIRED, WRONG_FACILITY, UNKNOWN_SESSION. |
| ParkingFacilityStatus | ACTIVE, INACTIVE, MAINTENANCE. |
| UserRole | USER. |
| DeviceCommandType | NONE, OPEN, KEEP_CLOSED. |
| CommandStatus | PENDING, DELIVERED. |

Table 5.10: Enumerations.

Three modelling details carry most of the lifecycle safety:

- createdAt is written when the user taps *start*. It is what the stale-session sweep measures, because a session that never reached a gate has no entry time.
- entryTimestamp is written only by the entry verdict. It is the only input to billing, so a session with no entry is provably free.
- exitGateOpenedAt is separate from exitTimestamp in intent, though in practice they hold the same instant. The grace window is measured from the gate opening, not from payment, so paying early and exiting late does not forfeit the re-open.

## 5.6 UI Design

The application is deliberately thin: it renders state, collects intent and carries messages. It does not decide fees or gate outcomes.

| Screen | Purpose and backend calls |
|---|---|
| Welcome | static onboarding |
| Register | `POST /auth/register`, then login |
| Login | `POST /auth/login`, then `GET /auth/me` |
| Dashboard | active session and vehicles (`/sessions/active`, `/vehicles`) |
| Discover | BLE scan, then `GET /facilities/{code}` |
| Start Session | vehicle picker and `POST /sessions` |
| Active Session | entry/exit BLE commands, live status |
| Payment | fee polling and `POST /sessions/{id}/pay` |
| Activity | `GET /sessions?page=0&size=50` |
| Profile | vehicle CRUD, sign-out, bench entry point |
| Gate Test | manual BLE commands for diagnostics |

Table 5.11: Screen design.

Navigation is a bottom tab bar over a native stack: the session screens are a stack because they depend on each other, while discovery, activity and profile are independent tabs. The payment screen polls the fee every two seconds while it is visible and stops on unmount, so a background screen never accumulates requests.

## 5.7 Security Design

Three kinds of caller exist, and each has its own credential. They are not interchangeable, and each is deliberately narrow.

The filter chain checks administration before routing and extracts JWTs before the web authorization rules [9]. A token is parsed only if it has three dot-separated segments, because device secrets arrive in the same `Authorization` header and must not be mistaken for tokens. A malformed or expired token leaves the request anonymous rather than failing inside the filter; the authorization rule then decides. Passwords and device secrets use the same one-way treatment, so a database dump yields neither a usable password nor a usable gate credential.

The token design follows the JSON Web Token standard [5]: a signed subject, an issue time and an expiry, with no sensitive claim in the payload. The threat model below follows the structure recommended for API security assessments [7] and records accepted risk rather than hiding it.

| Scheme | Used for | Storage |
|---|---|---|
| JWT bearer | Every user-facing endpoint. The subject is the user id; the token carries no role and no profile. | Stateless; signed with a base64 secret from the environment. |
| `X-Admin-Key` | Facility and device administration. Compared before the authorization rules run. | Environment variable; blank closes the surface. |
| Device bearer secret | `/gate/validate` and the command poll. 256 bits of entropy, returned once at registration. | BCrypt hash only. |

Table 5.12: Credential schemes.

| Threat | Mitigation or accepted risk (v1) |
|---|---|
| Rogue BLE client writes another user's session id | Gate re-validates with the backend; worst case is `keep_closed`. |
| Replay at the exit gate | Exit requires payment and the right state; `EXITED` is terminal apart from one bounded grace re-open. |
| Stolen JWT | Bearer-token risk bounded by the configured expiry; the client clears the token on 401/403. |
| Device secret extracted from a flashed board | Only the hash is stored, but physical access to a provisioned unit is accepted as game over. |
| BLE spoofing or man-in-the-middle | Accepted in version one; the backend is the trust anchor, not the link. |
| Fee manipulation | All arithmetic is server-side and uses only the backend clock. |
| Unauthenticated administration | Static key header, closed by default when the key is unset. |
| Payment fraud | The provider is a mock; no money moves in version one. |
| Tailgating | Fixed-timer close with no sensors; accepted and documented. |

Table 5.13: Threat model for the version-one posture.

# 6. Implementation

## 6.1 Technology Stack

Every technology in Table 6.1 was chosen for a reason that shows up in the design: a managed runtime for the transactional backend, a shared language for the phone and the API contract, and a toolchain that can be installed on a student laptop. Version and usage details for the stack are taken from the official documentation cited in the bibliography [8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20].

## 6.2 Module Description

### 6.2.1 Identity module

Registration hashes the password with BCrypt, ignores any client-supplied role and writes `USER`. Login verifies the hash and signs a JWT whose subject is the user id; the token carries no role and no profile, so it cannot be used to escalate privileges and profile changes do not invalidate it. A filter converts the bearer header into an authenticated principal whose name is that id. It parses only tokens with three dot-separated segments, because device secrets arrive in the same header; an invalid token is left anonymous and the authorization rules decide the response.

**Design note.** The authentication namespace is public by design — registration and login cannot require a token. `GET /auth/me` is meaningful only with a bearer token, and it is the mobile client's launch call that turns a persisted token back into a profile.

### 6.2.2 Vehicle module

Vehicle operations are owner-scoped. The controller takes the user id from the token, never from the body, and the service resolves the record and checks ownership before any mutation. Plate and class are the only attributes, which keeps the form short and matches what a gate actually needs.

### 6.2.3 Facility and administration module

Facilities are read by `facilityCode`, because that is what the gate advertises. The read model includes the pricing policy, so the application can show an hourly estimate without a second call. Creation, pricing and status changes require the administrator header. A new facility starts `INACTIVE` and must be activated before it accepts sessions, which prevents a half-configured site from taking parking sessions. A startup seeder inserts one active demo facility so a fresh stack is usable immediately.

| Layer | Technology | Version | Why |
|---|---|---|---|
| Backend | Spring Boot | 4.1 | Transactional REST, dependency injection and security out of the box. |
| Language | Java | 21 | Records, pattern matching and a mature JPA ecosystem. |
| Persistence | Spring Data JPA / Hibernate | Boot-managed | Repository interfaces, pessimistic locking, JSON column mapping. |
| Database | PostgreSQL | 16 | Strong constraints, `jsonb` for pricing, reliable locking. |
| Security | Spring Security + JJWT | 0.12.x | Filter chain and signed tokens without a custom crypto implementation. |
| Build | Maven (wrapper) | Boot-managed | Reproducible builds in the image; the wrapper removes a host install. |
| Testing | JUnit 5, Mockito, AssertJ | Boot-managed | Fast service tests with replaceable collaborators. |
| Coverage | JaCoCo | 0.8.13 | Line gate scoped to the service layer. |
| Mobile | Expo / React Native | 57 / 0.86 | One TypeScript codebase for iOS and Android; native module support for BLE. |
| Mobile language | TypeScript | 6.0 | The API envelope and DTOs are typed once and checked at build time. |
| BLE | `react-native-ble-plx` | 3.5 | Maintained GATT client with notify support; works in a development build. |
| Firmware | Arduino (ESP32 core) | esp32 board package | BLE and Wi-Fi libraries on the same board; no RTOS work required. |
| Firmware libs | ArduinoJson, ESP32Servo | 7.x, current | Small JSON documents, non-blocking servo control. |
| Packaging | Docker, Compose v2 | 24+ | One command to build, run and test the backend. |
| API testing | Postman / Newman | current | Collection-based contract checks for the auth controller. |
| Version control | Git / GitHub | current | Branch-based work and history. |

Table 6.1: Technology stack.

### 6.2.4 Session module

Session creation performs the full validation sequence in one transaction: the user must exist, the facility must exist and be active, the vehicle must exist and belong to the caller, and neither the user nor the vehicle may hold an active session. The read-then-write race is closed by locking the user row, so two concurrent requests serialise and the second sees the first session. The listing endpoint returns the caller's own sessions, newest first, with an optional status filter and a page size clamped to one hundred.

### 6.2.5 Billing and payment module

The live fee returns the frozen amount once one exists, refuses a session that has not entered, and otherwise multiplies the facility rate by whole elapsed seconds. The final fee is recomputed at payment and rounded up to the paisa, then written to `totalFee`. Payment locks the session row, so a double tap or a client retry sees `PAYMENT_PENDING` and is refused rather than charged twice. The mock provider implements a two-method interface and always succeeds; replacing it with a gateway means implementing the same interface.

### 6.2.6 Gate authorisation module

The gate service is the only place where entry and exit are decided. It checks the device, then the facility, then the session state, and it writes the timestamp that starts or ends billing. Granting an entry is the only place that writes `entryTimestamp`; granting an exit is the only place that writes `exitGateOpenedAt`. Because both methods are transactional, a verdict and the state it authorises cannot drift apart.

### 6.2.7 Device and command module

Registration generates 256 bits from a secure random source, returns the base64url secret once and stores only its BCrypt hash. Device authentication treats an unknown device, an unprovisioned device and a wrong secret identically, so a caller cannot probe which ids exist. Remote commands are rows rather than sockets: an operator queues `OPEN` or `KEEP_CLOSED` as `PENDING`, the gate's next poll takes the oldest pending row, marks it `DELIVERED` and returns it. The poll is the acknowledgement, so a gate that was offline still receives the command when it returns and never receives it twice.

### 6.2.8 Scheduled maintenance module

A scheduled component runs every minute and aborts `CREATED` sessions older than fifteen minutes, measured against `createdAt`. It is a thin trigger; the query and state change live in the session service so they can be tested without waiting for a scheduler. Both the interval and the age threshold are configuration properties.

### 6.2.9 Mobile application modules

The REST client owns the base URL, the token header and the envelope. It collapses the backend's semantics into three outcomes every screen can handle: success, a business refusal to display, and an authentication failure that clears the token and returns to login. The BLE client scans by service UUID, derives the facility code from the local name, caches one connection

per facility, subscribes to the status characteristic before writing, and resolves with the first notification that matches the session it is waiting for. Notifications for other sessions are ignored, which makes a concurrent remote command harmless to a waiting screen.

### 6.2.10 Firmware modules

The sketch is a loop around three interchangeable pieces: a transport (`WiFiBackendTransport` or `SerialBackendTransport`), an authentication service (`RemoteAuthService` or the bench-only `PermissiveAuthService`), and an actuator (`ServoGateActuator` or the legacy `LedGateActuator`). The transport is selected once at boot by sampling GPIO26. Above those interfaces there is one gate state machine, so changing the transport for a demonstration changes no timeout, no payload and no fail-closed behaviour.

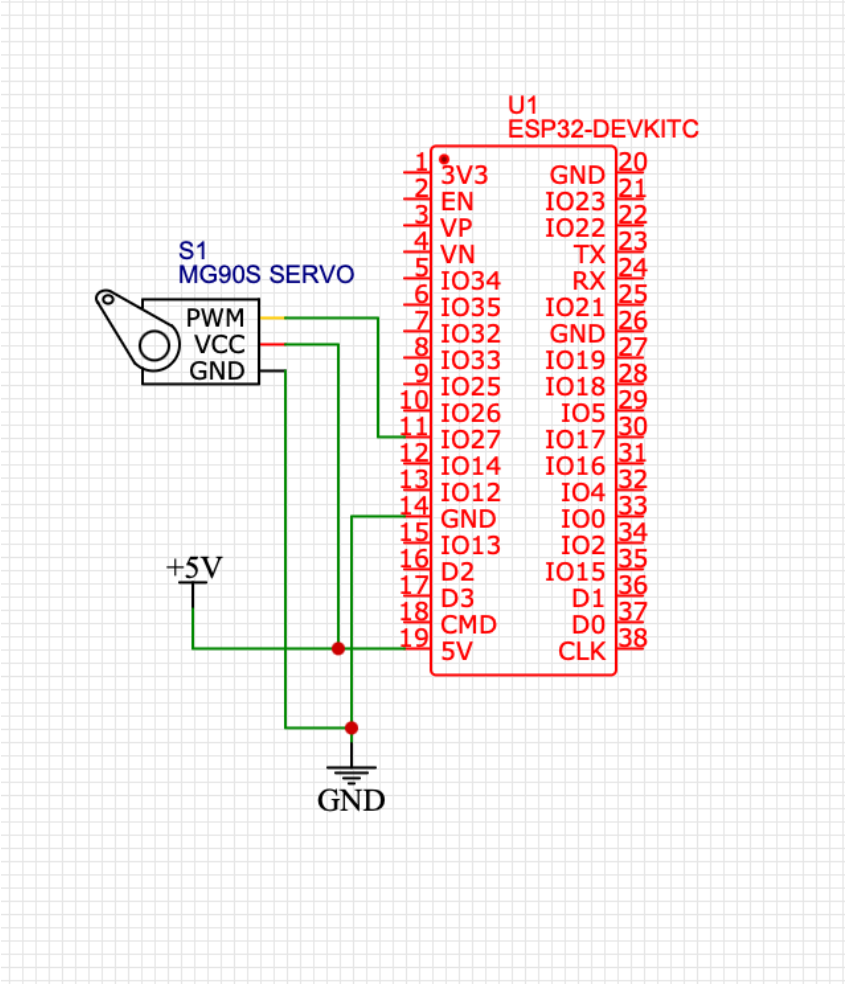



Figure 6.1: Gate wiring: servo signal on GPIO27, an external 5 V servo supply with a common ground and a decoupling capacitor, and the transport-select jumper on GPIO26.

## 6.3 Key Code Excerpts

The complete implementation is in the public repository; the excerpts below are the four pieces that carry the most design weight.

### 6.3.1 Session creation under a row lock

```java
@Transactional
public ResponseObject<UUID> createSession(SessionCreationRequestDto dto) {
    User user = userRepository.findByIdForUpdate(dto.userId()).orElse(null);
    if (user == null) return ResponseObject.failure("User does not exist.");

    ParkingFacility facility = facilityRepository
            .findByFacilityCode(dto.facilityCode()).orElse(null);
    if (facility == null)
        return ResponseObject.failure("Parking facility does not exist.");
    if (facility.getStatus() != ParkingFacilityStatus.ACTIVE)
        return ResponseObject.failure("Parking facility is not active.");

    Vehicle vehicle = vehicleRepository.findById(dto.vehicleId()).orElse(null);
    if (vehicle == null) return ResponseObject.failure("Vehicle does not exist.");
    if (!vehicle.getUser().getUserId().equals(user.getUserId()))
        return ResponseObject.failure("Vehicle does not belong to the user.");

    if (sessionRepository.existsByUser_UserIdAndSessionStatusIn(
            user.getUserId(), ACTIVE_STATUSES))
        return ResponseObject.failure("User already has an active parking session.");
    if (sessionRepository.existsByVehicle_VehicleIdAndSessionStatusIn(
            vehicle.getVehicleId(), ACTIVE_STATUSES))
        return ResponseObject.failure("Vehicle already has an active parking session."
    );

    ParkingSession session = SessionMapper
            .creationRequestToSession(dto, user, facility, vehicle);
    sessionRepository.save(session);
    return ResponseObject.success("Session successfully created", session.getSessionId
    ());
}
```

Listing 6.1: The active-session check is read-then-write, so the user row is locked for the transaction.

### 6.3.2 Payment freezes the fee

```java
@Transactional
public ResponseObject<PaymentReceiptDto> pay(UUID sessionId, UUID userId) {
    ParkingSession session = sessionRepository.findByIdForUpdate(sessionId)
            .orElse(null);
    if (session == null)
        return ResponseObject.failure("Parking session does not exist.");
    if (userId != null && !isOwnedBy(session, userId))
        return ResponseObject.failure("Parking session does not belong to the user.");
    if (session.getSessionStatus() != SessionStatus.PARKED)
        return ResponseObject.failure("Only a parked session can be paid for.");

    BigDecimal fee = calculateFinalFee(session);
    PaymentOutcome outcome = paymentService.charge(session.getSessionId(), fee);
    if (!outcome.successful())
        return ResponseObject.failure("Payment was not successful.");

    Instant paidAt = Instant.now();                // backend clock only
```

```
    session.setTotalFee(fee);
    session.setSessionStatus(SessionStatus.PAYMENT_PENDING);
    sessionRepository.save(session);
    // ... persist the Payment row and return the receipt
}
```

Listing 6.2: The session row is locked, the fee is frozen and the receipt records the backend's payment time.

### 6.3.3 Exit verdict with a grace re-open

```
switch (session.getSessionStatus()) {
    case ABORTED -> { return keepClosed(DenyReason.STALE_SESSION); }
    case EXITED -> {
        return withinExitGraceWindow(session)
                ? open()
                : keepClosed(DenyReason.STALE_SESSION);
    }
    case PAYMENT_PENDING -> {
        if (!hasSuccessfulPayment(session))
            return keepClosed(DenyReason.PAYMENT_REQUIRED);
        Instant exitedAt = Instant.now();
        session.setSessionStatus(SessionStatus.EXITED);
        session.setExitTimestamp(exitedAt);
        session.setExitGateOpenedAt(exitedAt);
        sessionRepository.save(session);
        return open();
    }
    // CREATED / PARKED - not paid yet
    default -> { return keepClosed(DenyReason.PAYMENT_REQUIRED); }
}
```

Listing 6.3: Exit is decided from state and payment; the grace window is measured from the gate opening.

### 6.3.4 BLE round trip in the application

```
const status = await this.withStatusNotification(device, sessionId, async () => {
  await device.writeCharacteristicWithResponseForService(
    SERVICE_UUID,
    COMMAND_CHAR_UUID,
    Buffer.from(JSON.stringify({ sessionId, intent }), "utf8").toString("base64")
  );
});

if (!status) {
  throw new Error(
    `Gate ${facilityCode} did not answer over BLE. Move closer and check power.`);
}
return { result: status.result, reason: status.reason || "NONE" };
```

Listing 6.4: Subscribe before write, filter by session id, and give the whole round trip a fifteen-second budget.

### 6.3.5 Firmware transport selection

```
static BackendTransport *selectTransport() {
    pinMode(NETWORK_MODE_PIN, INPUT_PULLUP);
    delay(10);
    if (digitalRead(NETWORK_MODE_PIN) == LOW) return &serialTransport;
    return &wifiTransport;                        // production path
}
```

Listing 6.5: One pin decides the transport at boot; all gate logic above it is shared.

# 7. Software Tools Used

Five tools did real work in this project rather than merely hosting it. For each, this chapter states the purpose, the specific use in DWARA, and the artefact that use produced. Screenshots are referenced through the image registry, so a replacement capture only needs a path change in `images.tex`.

## 7.1 Git and GitHub

**Purpose.** Distributed version control and a remote home for the repository, including history, branching and review.

**Use in DWARA.** The repository holds all three deliverables so that a change to the API and the client that consumes it can be reviewed together. Work was done on feature branches and merged into the integration branch (`finish-backend`); the history records the progression from backend scaffolding to the complete system. Commit messages state the reason for a change, which makes the history usable as a design log. Git also enforces hygiene: `.env`, `target/` and the generated `reports/` tree are ignored, so no secret or build artefact enters version control.

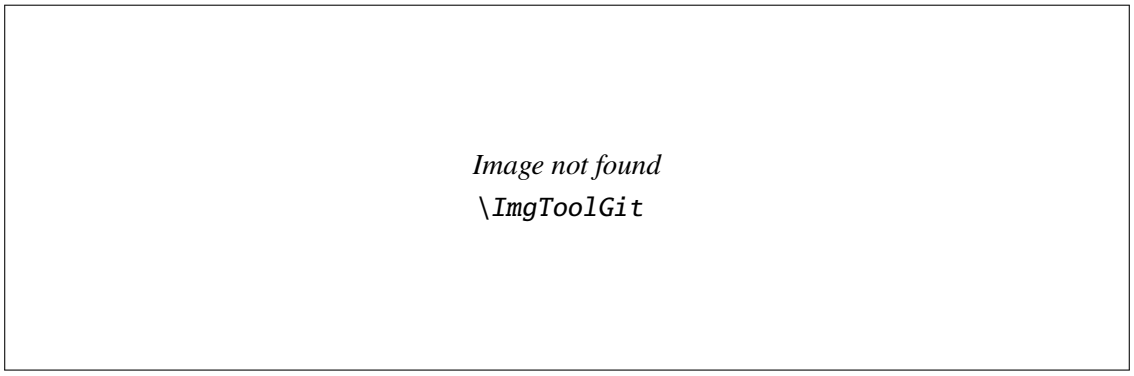



Figure 7.1: Git history and branch state in the repository.

## 7.2 Docker and Docker Compose

**Purpose.** Reproducible build and runtime environments, and a single command that brings up the database, the application and the test runner.

**Use in DWARA.** The backend is built in a multi-stage image: a Maven stage resolves dependencies and packages `target/app.jar`, a runtime stage runs it as a non-root user, and a third stage reuses the build image as the test runner. Compose defines PostgreSQL with a health check, the backend with a health check and a dependency on the database, and a test-only

profile that runs `mvn verify` and writes reports back to the host. The test project uses its own compose project name, ports and volume, so a verification run never disturbs a development stack.

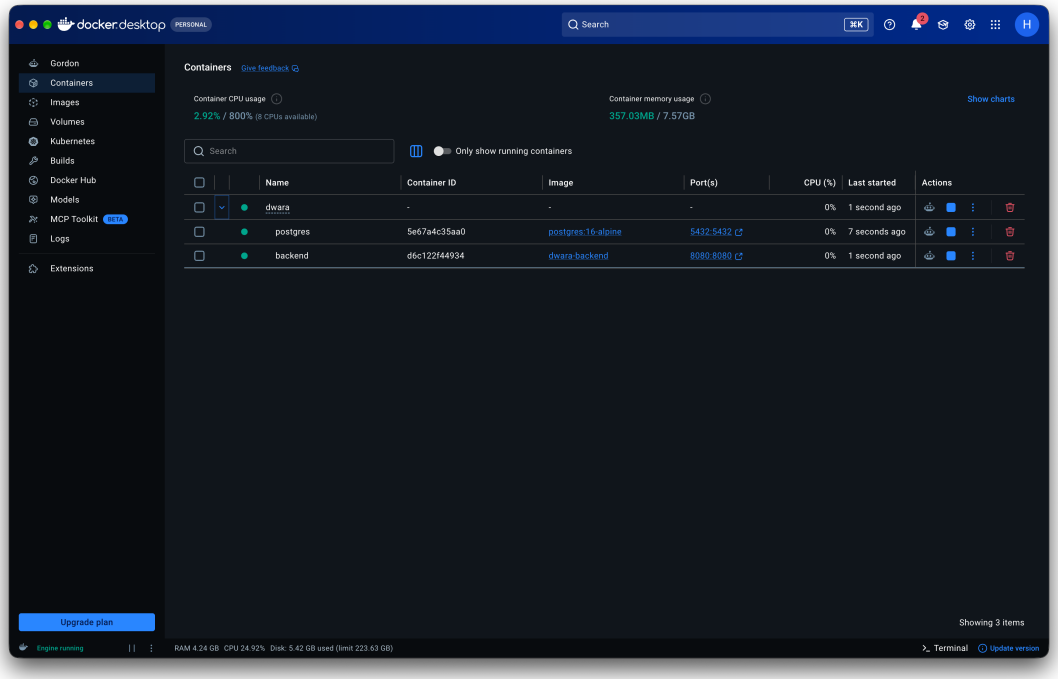



Figure 7.2: Compose stack in operation and the test workflow output.

## 7.3 Postman

**Purpose.** Request-level API exploration and a durable, executable record of the API contract.

**Use in Dwara.** The collection stored at `src/test/postman/dwara-auth.postman_collection.json` concentrates on the authentication controller: registration, duplicate registration, login, a wrong password, the authenticated profile call and the unauthenticated profile call. It contains six requests and nineteen assertions, and it chains state: the registration request generates a unique e-mail and stores the returned user id, the login request stores the token, and the profile request proves that the token resolves to the same account. A collection pre-request script makes the suite repeatable, so the same collection can be run from the Postman runner or from Newman without leaving duplicate accounts behind.

## 7.4 Visual Studio Code

**Purpose.** One editor for Java, TypeScript, shell and Arduino sources, with language support that catches errors before a build.

**Use in Dwara.** VS Code hosted the backend, the Expo application and the firmware side by side. The Java extension provided navigation across controller, service and repository layers; the TypeScript server enforced the types shared between the API client and the screens; the integrated terminal ran the Maven wrapper, the Compose stack and `arduino-cli`. Keeping all

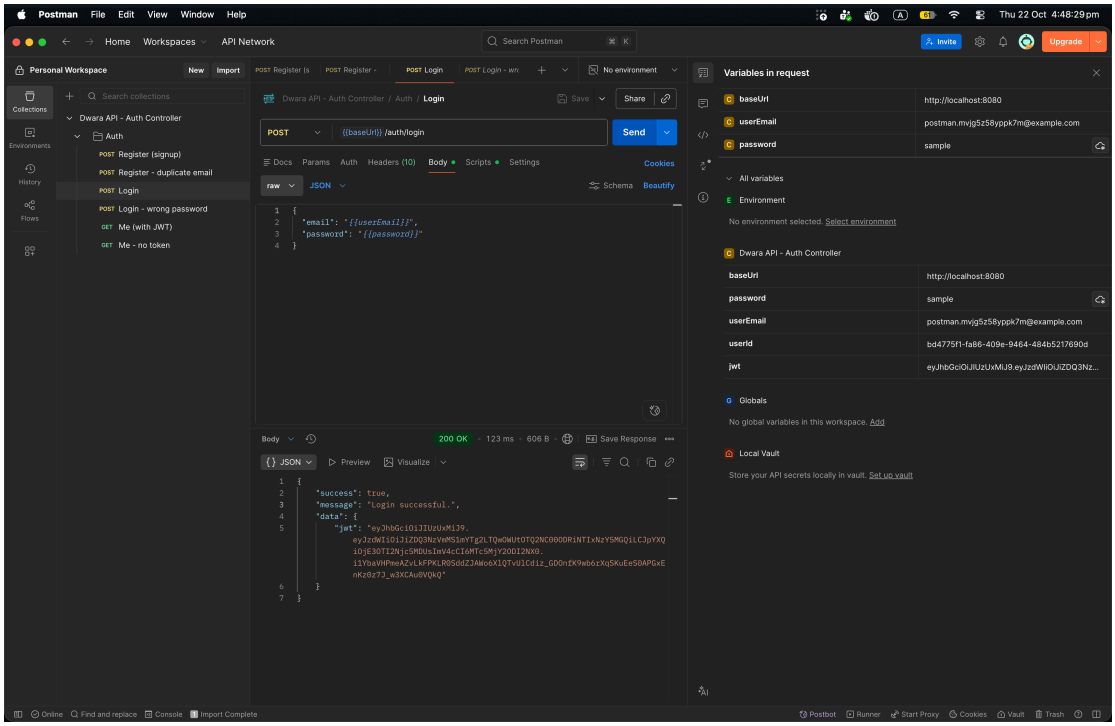



Figure 7.3: The authentication collection with its assertions in the Postman runner.

three surfaces in one workspace made it practical to change a DTO and its TypeScript type in the same edit.

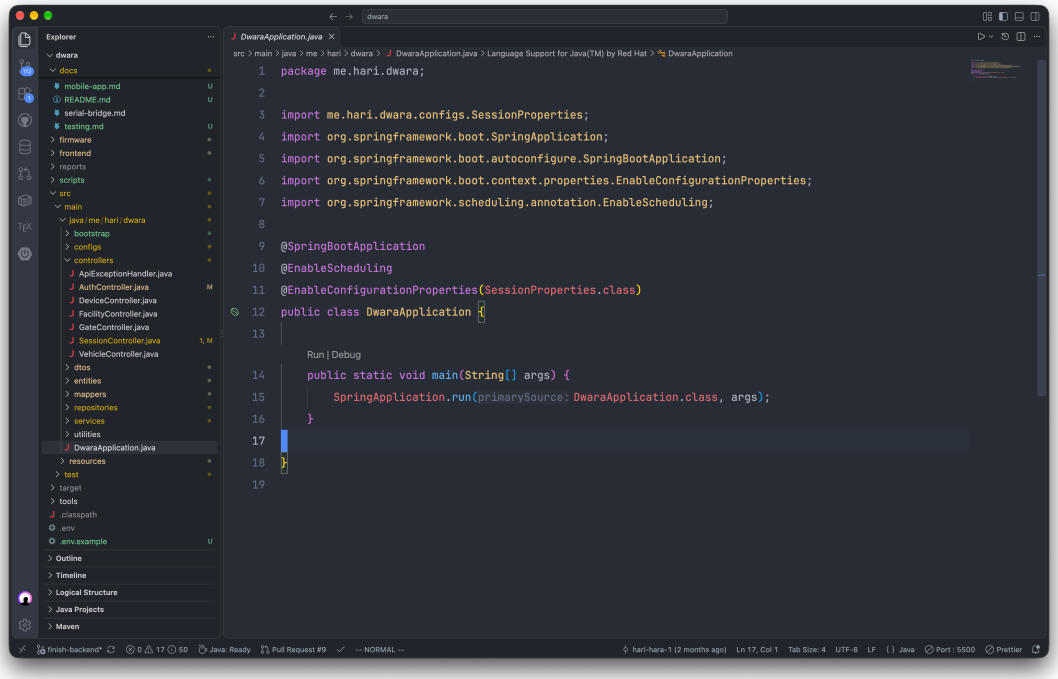



Figure 7.4: The monorepo open in VS Code, with the backend and the application in one workspace.

## 7.5 Arduino CLI and Arduino IDE

**Purpose.** Compile and flash the ESP32 firmware, and read the serial monitor during bench testing.

**Use in DWARA.** The firmware is compiled headlessly with `arduino-cli` against the ESP32 core and the Huge APP partition scheme, which makes a compile check possible without opening a GUI:

```
arduino-cli compile --fqbn esp32:esp32:esp32:PartitionScheme=huge_app \
  firmware/dwara-gate
```

Listing 7.1: Firmware compile check used after every firmware change.

The IDE was used for flashing and for the serial monitor at 115200 baud, where the boot banner reports the selected transport, the servo pin and the advertised BLE name. The same monitor carries the framed serial-bridge traffic used in the Wi-Fi-less demonstration.

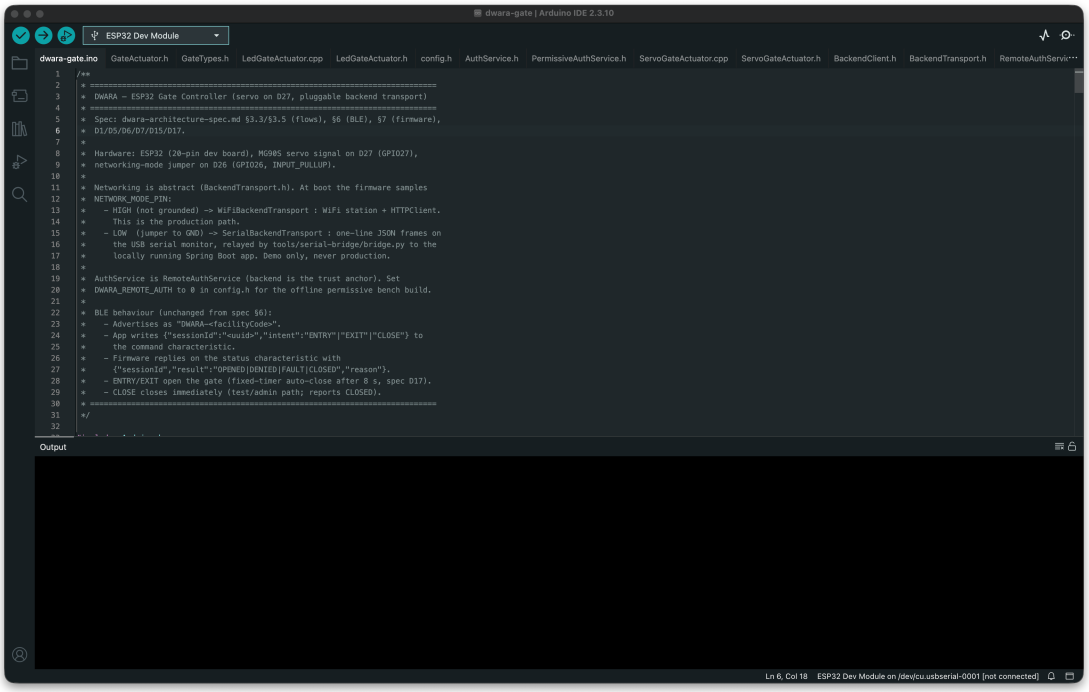



Figure 7.5: Firmware compile and the gate's serial output during a bench run.

## 7.6 Summary

| Tool | Purpose | Phase of SDLC | Usage in this project |
|---|---|---|---|
| Git / GitHub | Version control and collaboration | All phases | Monorepo, feature branches, reviewable history, ignored secrets and build output |
| Docker / Compose | Reproducible environments | Implementation, testing, deployment | Multi-stage backend image, PostgreSQL service, test profile, bind-mounted reports |
| Postman | API contract testing | Testing | Six-request authentication collection with nineteen chained assertions; runnable in the app or Newman |
| VS Code | Editing and static analysis | Design, implementation, testing | Java navigation, TypeScript checking, integrated terminal for Maven, Compose and Arduino CLI |
| Arduino CLI / IDE | Firmware build and bench diagnostics | Implementation, testing | Headless compile check, flashing, serial monitor and serial-bridge traffic |

Table 7.1: Tool summary.

# 8. Testing

## 8.1 Test Plan and Strategy

Testing followed the shape of the system: business rules were tested below HTTP, contracts were tested at HTTP, and the pieces that cannot run on a laptop were verified by compilation and by a documented bench procedure.

| Level | Scope and technique |
|---|---|
| Service unit tests | Eleven classes cover the service layer with JUnit 5 and Mockito. Repositories, the payment provider, the gate client and the clock boundaries are replaced by doubles, so a rule can be tested without a database or hardware. |
| Context test | One Spring Boot context load against the test profile, which proves the application wires up with the real configuration. |
| Persistence test | One repository test for the payment lookup query, executed against PostgreSQL rather than an in-memory substitute. |
| DTO and configuration tests | The response envelope serialises its payload as `data`, and the session timing properties bind ISO-8601 durations. |
| Contract tests | A Postman collection exercises the authentication controller over HTTP, including the refusal paths. |
| Workflow tests | The Compose test workflow is itself verified: the production image must become healthy before the suite starts, and reports must survive a failing run. |
| Static checks | TypeScript type-check for the mobile application and a headless compile of the firmware for the ESP32 target. |
| Bench tests | Servo sweep, stall watchdog, BLE round trip and serial bridge, executed manually on hardware with a documented procedure. |

Table 8.1: Test levels.

### Environment

Tests run in the `test` container of the Compose project. The container waits for PostgreSQL to become healthy and for the production image to report healthy at `/actuator/health`; the latter is a readiness gate, not a smoke test, and exists to prove that the deployment configuration boots. The test profile points at a separate database, `dwara_test`, created by an initialisation script, and uses `create-drop` so every run starts from an empty schema. JWT secret, token lifetime and administrator key are generated per run and injected as environment variables; the committed values in the test profile are explicitly test-scope.

No H2 and no Testcontainers are used. Both would have been convenient and both would have tested a different database from the one that ships.

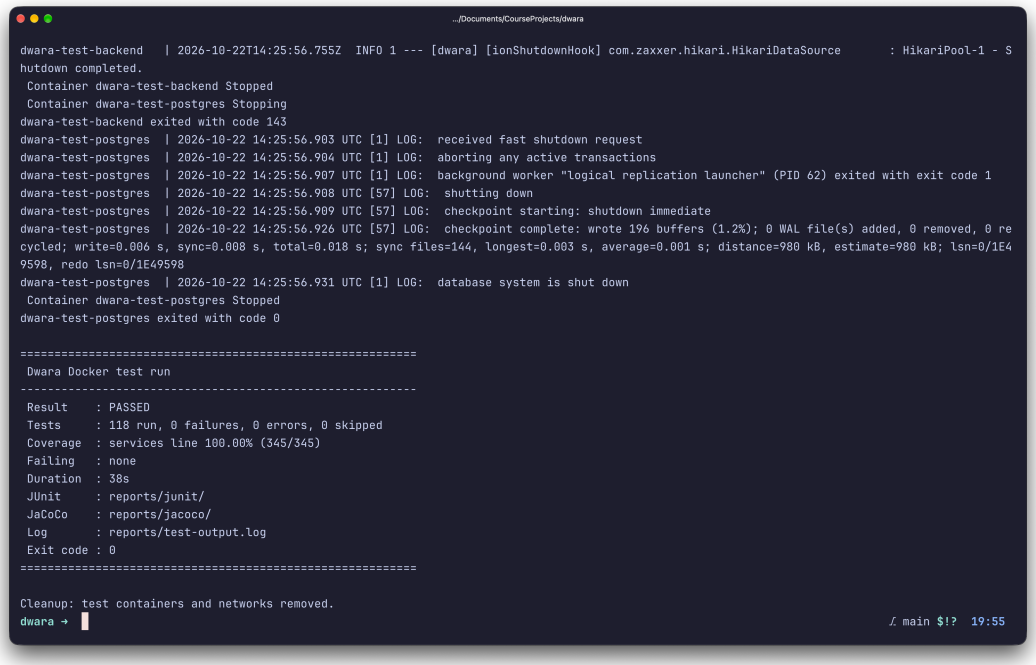



Figure 8.1: The verification pipeline. A failing suite still produces both report trees, because collection happens after the exit status is captured.

### Coverage policy

JaCoCo is scoped to `me/hari/dwara/services/**` and the gate is a `COVEREDRATIO` of 1.00 on lines, enforced in the `verify` phase. Scoping is deliberate: entities, DTOs and mappers are mostly generated or declarative, and including them would dilute the number without adding evidence. Enforcing the gate in `verify` rather than in `package` means the Docker image build is not blocked by coverage, while the test workflow is.

## 8.2 Test Cases

Table 8.2 lists the representative cases, including the refusals that matter most. “Pass” means the expected and observed outputs matched in the automated suite; the two firmware cases are marked as requiring hardware because their evidence is a documented bench procedure rather than an executed test in this environment.

## 8.3 Test Results and Bug Log

### Automated suite

The suite contains 118 tests in 15 classes, all of which pass. The distribution follows the risk: the session service and the gate service carry the most rules and therefore the most tests.

| ID | Module | Input / action | Expected output | Observed output | Status |
|---|---|---|---|---|---|
| TC-01 | Auth | register with a new e-mail | `success: true`, `data.userId` present | envelope with generated id and no password field | Pass |
| TC-02 | Auth | register the same e-mail twice | `success: false`, "User already exists." | refusal in the envelope | Pass |
| TC-03 | Auth | login with correct credentials | `success: true`, three-part JWT | JWT returned and accepted by `/auth/me` | Pass |
| TC-04 | Auth | login with a wrong password | refusal, no token | refusal, no token | Pass |
| TC-05 | Auth | `/auth/me` with a valid token | profile of the same account | matching user id and e-mail | Pass |
| TC-06 | Session | start a session at an active facility | `CREATED`, UUID returned | `CREATED` with a UUID | Pass |
| TC-07 | Session | start twice with the same user | refused, first session kept | "User already has an active parking session." | Pass |
| TC-08 | Session | same user, second vehicle | refused (per-user rule) | refusal | Pass |
| TC-09 | Billing | fee of a parked session | rate × elapsed seconds | live amount increases per second | Pass |
| TC-10 | Billing | pay a parked session | `PAYMENT_PENDING`, frozen amount, receipt | receipt with `MOCK-` reference | Pass |
| TC-11 | Billing | pay the same session twice | refused, single payment row | "Only a parked session can be paid for." | Pass |
| TC-12 | Gate | exit before payment | `keep_closed` / `PAYMENT_REQUIRED` | same verdict and reason | Pass |
| TC-13 | Gate | exit after payment | `open`, session `EXITED` | same | Pass |
| TC-14 | Gate | re-approach after exit | `open` inside the grace window, `keep_closed` outside | both boundaries honoured | Pass |
| TC-15 | Sweep | session older than the stale threshold | `ABORTED` | aborted, slot released | Pass |
| TC-16 | Device | validate with a wrong device secret | 401 | 401 with the standard envelope | Pass |
| TC-17 | Admin | create a facility without the admin key | 401 | 401 | Pass |
| TC-18 | History | list sessions with page and status | own sessions only, newest first, paging metadata | matching page metadata | Pass |
| TC-19 | Firmware | malformed BLE payload | `DENIED/MALFORMED_COMMAND` | documented bench procedure | Hardware |
| TC-20 | Firmware | blocked servo during a sweep | `FAULT`, PWM released | documented bench procedure | Hardware |

Table 8.2: Representative test cases. The complete suite is enumerated in `reports/junit` after any run.

| Test class | Tests | Area |
|---|---|---|
| `ParkingSessionServiceTest` | 31 | lifecycle, billing, anti-abuse, history |
| `GateControllerServiceImplTest` | 21 | entry/exit verdicts, grace window |
| `VehicleServiceTest` | 13 | ownership-scoped CRUD |
| `ParkingFacilityServiceTest` | 12 | creation, pricing, status |
| `UserServiceTest` | 9 | registration, login, profile |
| `GateCommandServiceTest` | 7 | queue, poll and acknowledgement |
| `DeviceAuthenticationServiceTest` | 7 | device secret verification |
| `PollingGateControllerClientTest` | 6 | gate client behaviour |
| `DeviceProvisioningServiceImplTest` | 3 | secret generation and hashing |
| `ScheduledSessionAborterTest` | 2 | sweep trigger |
| `MockPaymentServiceTest` | 2 | provider contract |
| `SessionPropertiesTest` | 2 | duration binding |
| `PaymentRepositoryQueryTest` | 1 | payment lookup query |
| `DwaraApplicationTests` | 1 | context load |
| `ResponseObjectTest` | 1 | envelope serialisation |
| **Total** | **118** | |

Table 8.3: Automated test inventory.

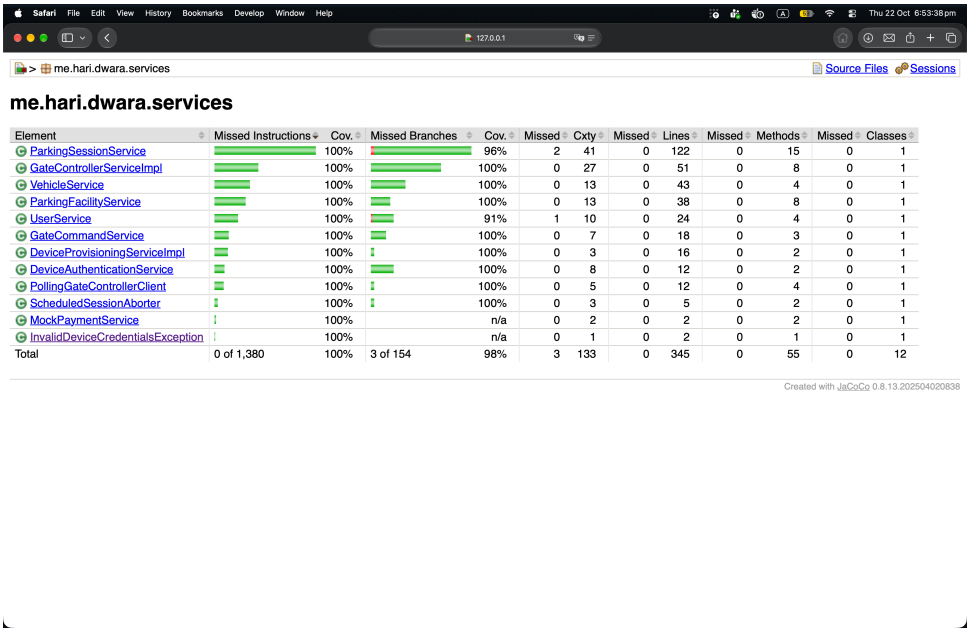



Figure 8.2: JaCoCo report for the service package: 100% line coverage under the enforced gate.

| Check | Command or artefact | Result |
|---|---|---|
| Backend suite | `mvn verify` in the test container | 118 tests, 0 failures, 0 errors |
| Coverage gate | JaCoCo `check`, service package, line ratio | 100%, gate passed |
| API contract | Postman collection, six requests, nineteen assertions | positive and refusal paths pass; the unauthenticated probe records the public-namespace boundary |
| Mobile types | `npm --prefix frontend run typecheck` | clean |
| Firmware build | `arduino-cli compile` for ESP32, Huge APP | compiles |
| Health gate | `curl /actuator/health` | `{"status":"UP"}` |

Table 8.4: Verification summary.

## Defect log

The defects below were found during implementation, each by a specific check, and closed before the final verification. They are recorded because the fix in each case is now part of the design rather than a patch.

## Interpretation

Three observations are worth drawing from the numbers. First, the rule-heavy classes attract the tests: two services account for 52 of 118 tests, which matches where the failure modes live. Second, the coverage gate changed behaviour rather than merely reporting it — a new branch in a service could not be merged without a test, so coverage stayed at 100% without a cleanup phase. Third, the checks that caught the most consequential defects were not unit tests but integration-shaped ones: the context load caught the reserved-word problem, and the container workflow caught the health-gate dependency order.

| ID | Symptom | Root cause and resolution | Status |
|---|---|---|---|
| D-01 | Application failed at startup on schema creation | `user` is reserved in PostgreSQL; the entity is mapped to `users`. | Closed |
| D-02 | Device calls were rejected as malformed tokens | The JWT filter parsed any bearer value; it now parses only three-segment tokens and leaves the rest anonymous. | Closed |
| D-03 | Gate sometimes never reported an outcome | The client subscribed after writing; it now subscribes first and filters by session id. | Closed |
| D-04 | Grace re-open expired while a user was still walking to the gate | The window was measured from payment; `exitGateOpenedAt` was added and the window is measured from the gate opening. | Closed |
| D-05 | Two concurrent start requests produced two sessions | Read-then-write race; the user row is now locked for the transaction. | Closed |
| D-06 | A retried payment could charge twice | No state guard under concurrency; the session row is now locked and the state is re-checked. | Closed |
| D-07 | A jammed servo stayed energised | The watchdog closed the angle but kept the PWM attached; it now detaches the signal and latches `FAULT`. | Closed |
| D-08 | A remote poll could delay an auto-close | The poller ran during movement; it now polls only while the gate is closed and the actuator is idle. | Closed |

Table 8.5: Defect log. Each entry is closed and covered by a regression test or a documented bench check.

# 9. Results and Discussion

## 9.1 Output of the Working System

The application is usable end to end against the composed backend. The screens below show the driver's path from discovery to a closed history entry; every value on them comes from the backend, not from a fixture.

### 9.1.1 End-to-end walkthrough

The walkthrough below was executed against the development stack with a seeded demo facility and one registered gate. Each step lists the observable result rather than the implementation behind it.

| Step | Action | Observed result |
|---|---|---|
| 1 | Register, then sign in | Account created; token stored; dashboard shown |
| 2 | Add a vehicle | Plate and class accepted; vehicle appears on the dashboard |
| 3 | Scan for gates | `DWARA-FAC-0001` listed; facility detail shows the per-second rates |
| 4 | Start a session | `CREATED`; the identifier is shown and the entry button becomes available |
| 5 | Open the entry gate | Backend returns `open`; the servo sweeps; status becomes `PARKED` with an entry timestamp |
| 6 | Wait and watch the fee | Amount increases once per second at the facility rate |
| 7 | Pay | Receipt with a `MOCK-` reference; amount frozen; session `PAYMENT_PENDING` |
| 8 | Open the exit gate | Backend verifies the payment; gate opens; session `EXITED` |
| 9 | Open the activity screen | The finished session appears with its frozen total |
| 10 | Queue a remote `OPEN` (operator) | Gate executes on its next poll and reports `sessionId: "remote"` |

Table 9.1: End-to-end walkthrough.

Two behaviours in the walkthrough deserve emphasis because they demonstrate that the architecture is doing real work. In step 5 the application cannot open the gate: it only delivers the request, and the state change is visible because the backend recorded it. In step 8 the exit is granted by payment state, not by the phone, so the same session cannot be used to re-enter the facility.

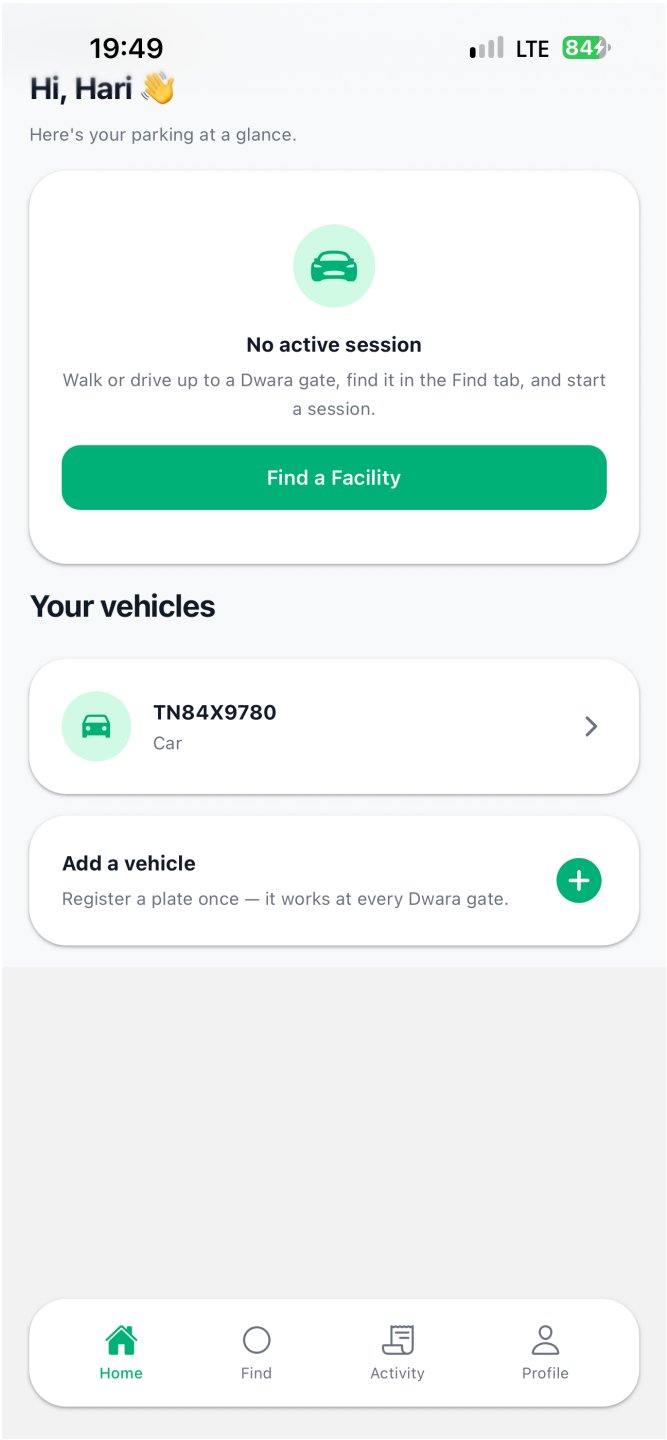



Figure 9.1: Dashboard after sign-in: the active session and the vehicles held by the signed-in account.

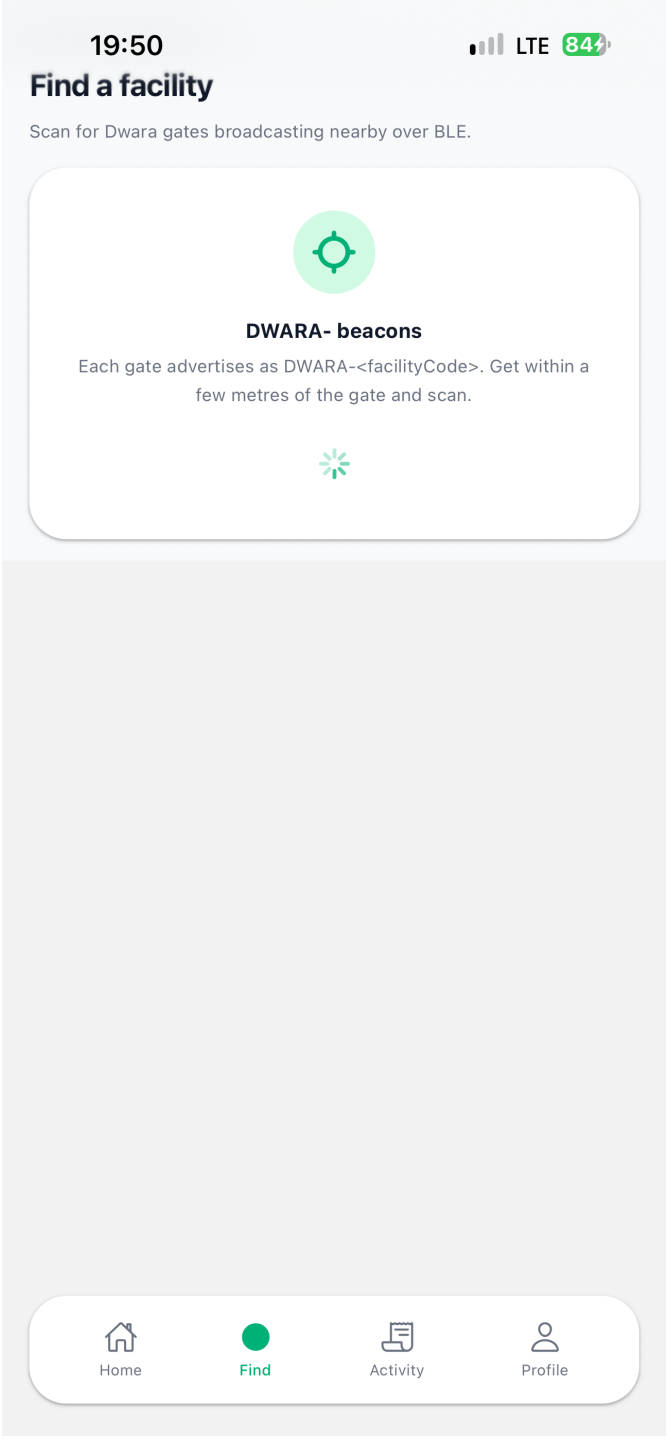



Figure 9.2: Discovery: the gate advertising `DWARA-FAC-0001` appears in the scan, and its detail is fetched from the backend by facility code.

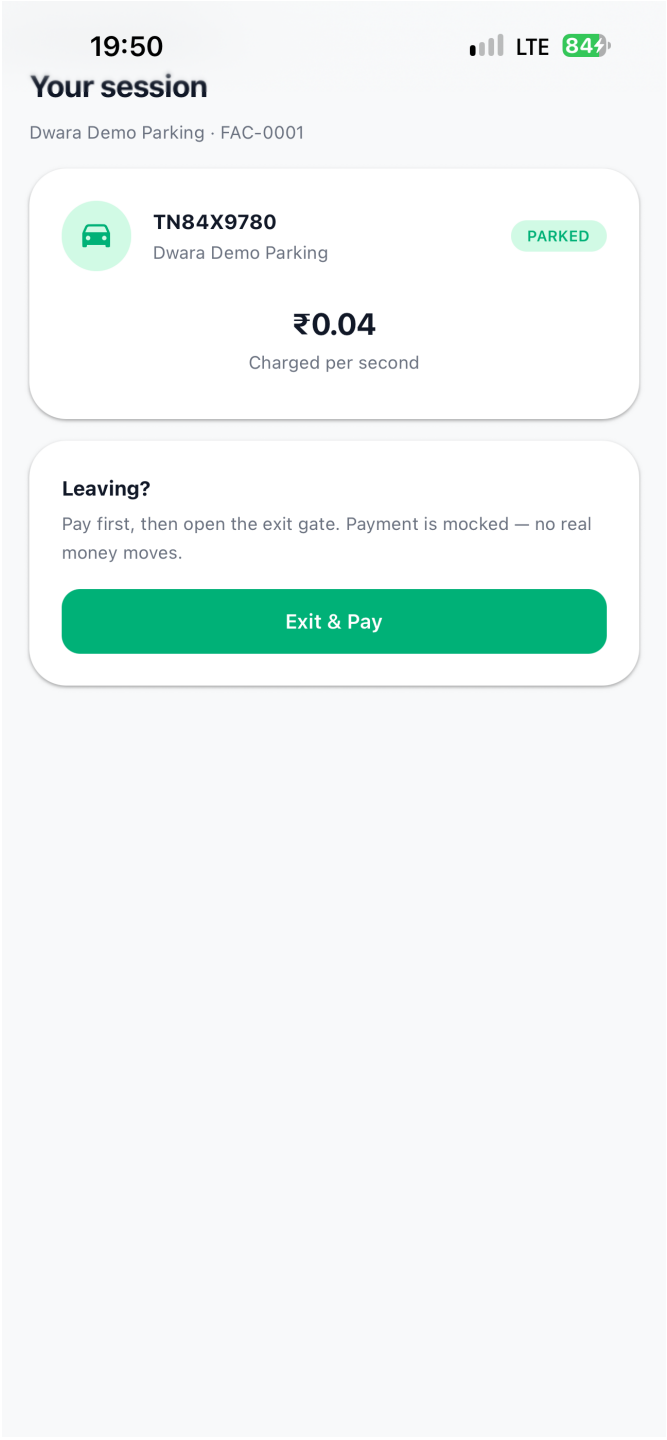



Figure 9.3: Active session: the entry gate button, the live status and the running fee.

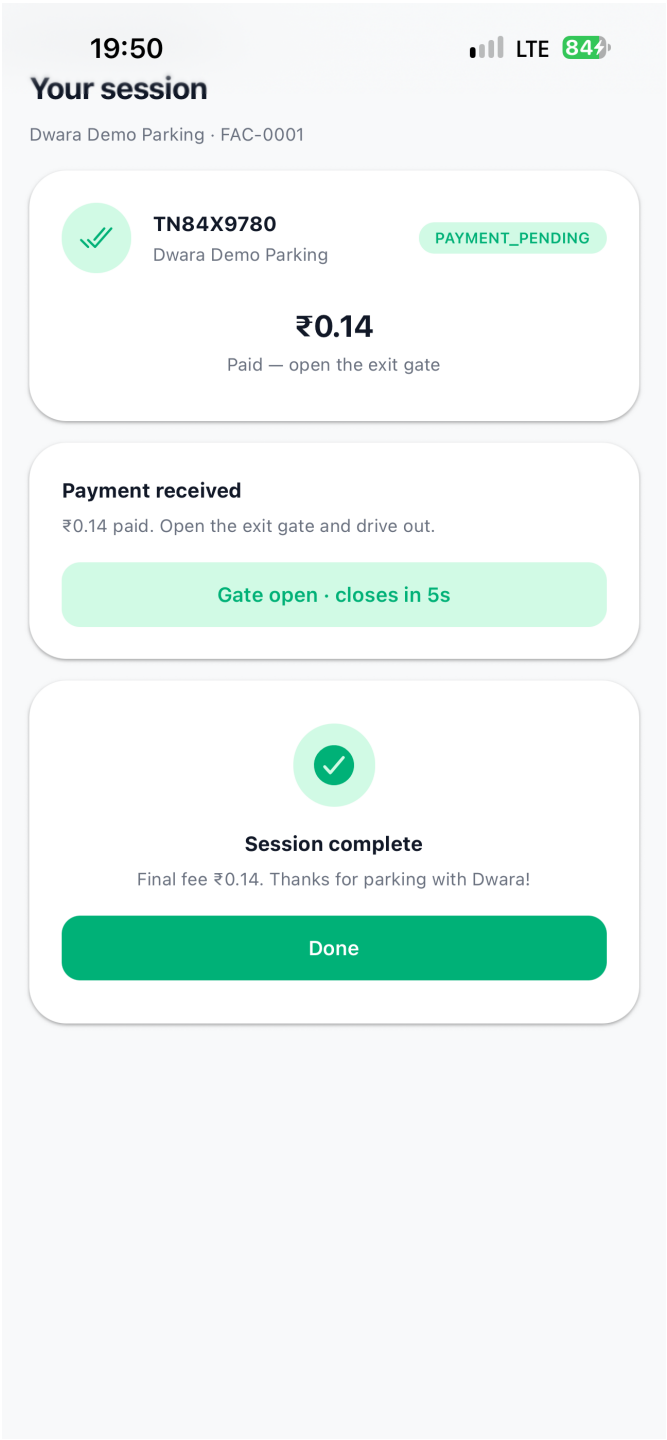



Figure 9.4: Payment: the frozen amount after a successful mock payment, with the exit gate now available.

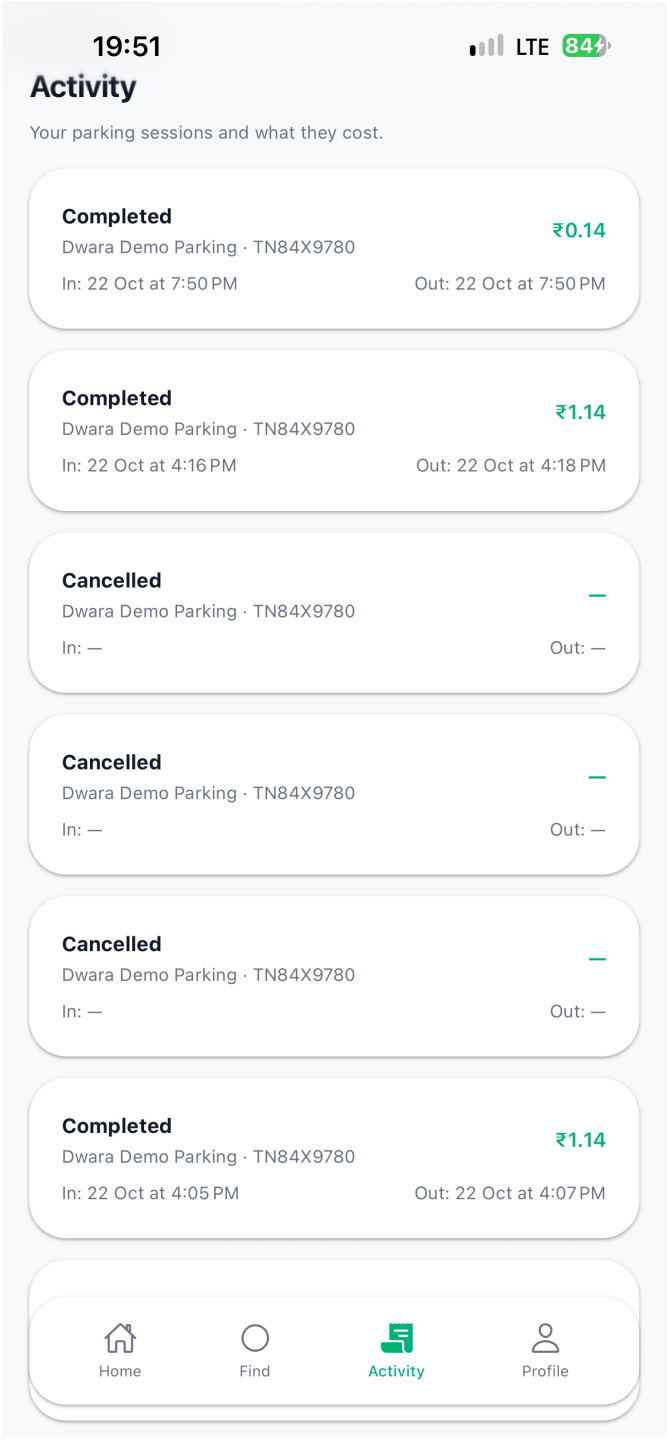



Figure 9.5: Activity: the finished session listed from the paged history endpoint.

| Objective | Evidence | Outcome |
|---|---|---|
| Frictionless discovery | The gate advertises `DWARA-<facilityCode>`; the application lists it from a scan and resolves details through `GET /facilities/{code}`. | Met |
| Server-authoritative control | Every entry and exit is a `POST /gate/validate` decision; the firmware has no rule that can open a gate on its own, and fails closed on transport errors. | Met |
| Auditable per-second billing | Amounts are computed from `entryTimestamp` and the facility rate, frozen at payment, and returned with a receipt; the client never supplies a time or an amount. | Met |
| Safe lifecycle | One active session per user and per vehicle, guarded by a row lock; stale sessions reaped; every illegal transition has a reason code and a test. | Met |
| Three-surface deliverable | Backend, Expo application and ESP32 firmware in one repository, with a shared configuration model and one Compose stack. | Met |
| Reproducible verification | One command builds, boots, tests and publishes reports; 118 tests pass under a 100% service-line coverage gate. | Met |

Table 9.2: Objectives against evidence.

## 9.2 Analysis Against Objectives

The only objectives that are met at the software boundary rather than on hardware are the firmware behaviours. The sketch compiles for the ESP32 target and the serial bypass allows the full backend flow to be exercised without Wi-Fi, but the servo sweep, the stall watchdog and the BLE round trip on a physical gate are documented bench procedures, not results measured in this environment. That is a limitation of the evaluation, not of the design, and it is the first item in the future-work list.

### Measured observations

Table 8.4 in the previous chapter records the verification results. Three numbers are worth reading as outcomes rather than as counts:

- **118 automated tests, 0 failures, 100% service-line coverage.** The coverage gate is enforced in `verify`, so the figure cannot drift silently.
- **Sub-second backend responses in the local stack.** The authentication contract run recorded responses between roughly 6 ms and 155 ms, with the password-hashing calls in registration and login at the slow end of that range.
- **Deterministic gate timing.** The open hold is a fixed 8 s and the settle window 800 ms, so the application's 15 s round-trip budget leaves ample margin even when the backend is slow.

### Discussion

The most useful result is negative: the system works without any component that identifies a vehicle. A phone account, a session identifier and a server-side state machine were sufficient

to authorise entry and exit, and the only hardware at the site is a microcontroller and a hobby servo. That is a meaningful cost reduction for a small facility, where a camera or an RFID reader per lane would dominate the budget.

The second result is that centralising the decision simplified the edge. The firmware contains no pricing, no payment state and no notion of a user; it contains a timeout, a verdict and a servo. When the BLE client was fixed during development, the firmware did not change; when the grace window moved from payment time to gate time, the firmware did not change either. The cost of that simplification is a hard dependency on the backend at the moment of entry and exit, and a deliberate one: the gate fails closed, and the operator has a remote-open path for the post-payment case.

The third result concerns testability. Because the payment provider, the gate client and the actuator are interfaces, the rule-heavy code is testable without a database or a servo. The classes with the most tests are the two that decide money and access, which is where defects would be most expensive.

The design also has costs that should be stated plainly. A fixed-timer gate cannot detect tailgating; plaintext BLE means the session identifier is visible to anything in range; a single gate per facility means one queue serves both directions; and the mock payment provider means the settlement path has never seen a real failure. Each of these is a deliberate version-one boundary, and each has a concrete replacement in the next section.

# 10. Conclusion and Future Enhancements

## 10.1 Conclusion

DWARA set out to remove the dedicated identity hardware from a parking gate and replace it with a phone that the driver already owns. The delivered system does that without weakening the access decision: the phone carries a session identifier, the gate asks the backend, and the backend alone decides. Entry starts the billing clock, payment freezes the amount, and the exit verdict checks the payment rather than the clock. A camera, an RFID reader, a ticket dispenser and a vehicle sensor are all absent, and the entire site is one ESP32 and one servo.

The work produced four artefacts that can be evaluated separately. The backend is a layered Spring Boot application with three credential schemes, a transactional session lifecycle, per-second billing and a device-command queue. The mobile application is a typed Expo client that owns no business rules and displays the backend's own messages. The firmware is an Arduino sketch whose transports and actuator sit behind interfaces, so the same gate logic runs over Wi-Fi, over a serial demonstration bridge, or against an LED stand-in. The verification workflow is a Compose profile that boots the real image as a readiness gate, runs 118 tests under a 100% service-line coverage gate, and publishes JUnit and JaCoCo reports even when the run fails.

Three conclusions are worth carrying forward. First, *the correct place for the access decision is the server*, and moving it there makes the edge dramatically simpler rather than more complex. Second, *duration and payment are the only inputs the exit gate needs*; once that is true, a single gate can serve both directions and the facility needs no second mechanism. Third, *reproducible verification is a feature*: the container workflow turned every later change into a checkable one and made the coverage figure a constraint rather than a report.

The system is complete for its stated scope. What remains is not unfinished work but the next layer of hardening: real payments, transport security, richer gate hardware and the operational surfaces that a commercial deployment would require.

## 10.2 Future Enhancements

The ordering follows one rule: remove the assumptions that would be unacceptable in production before adding capability. A real payment provider and transport security address the two places where the current system is explicitly a prototype. Hardware validation converts documented procedures into measured evidence. Everything else expands the product; none of it changes the architecture, because the decision that matters is already in the right place.

| Enhancement | Description and why it matters | Priority |
|---|---|---|
| Real payment gateway | Replace the mock provider with an implementation of `PaymentService` that supports authorisation, failure and refunds. The session lifecycle already models the states a gateway needs. | High |
| Transport security | Serve the API over TLS and move the gate to HTTPS; consider BLE pairing with an authenticated session token instead of a plaintext identifier. | High |
| Hardware validation | Execute and record the servo, watchdog and BLE bench procedures on a physical gate, and add an automated end-to-end test with a development board on the bench. | High |
| Escalating enforcement | Add sensors, a beam or a loop detector so the gate can detect a tailgating vehicle and refuse to close on an obstruction. | Medium |
| Multiple gates and lanes | Generalise the device model to entry and exit gates per facility and to more than one servo, with a lane identifier in the verdict. | Medium |
| Operator console | A small web surface for facilities, pricing, device rotation and live gate state, replacing curl-style administration. | Medium |
| Device secret rotation | A rotation endpoint that returns a new secret once, invalidates the old hash immediately and records the event. | Medium |
| Over-the-air updates | Signed firmware updates for the gate, with rollback, so a provisioned site does not need physical access to change behaviour. | Low |
| Occupancy and analytics | Optional bay sensors or camera counts to expose utilisation and revenue trends without making access depend on them. | Low |
| Background BLE and wallet cards | iOS background operation and a fallback NFC or card credential for drivers without the application. | Low |

Table 10.1: Enhancement roadmap, ordered by the risk each item removes.

# References


[1] A. Mackey, P. Spachos, and K. N. Plataniotis, "Smart parking system based on Bluetooth low energy beacons with particle filtering," *IEEE Systems Journal*, 2020, doi: 10.1109/JSYST.2020.2968883.

[2] M. R. Alam *et al.*, "A survey on IoT driven smart parking management system," *IEEE Access*, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10292845

[3] C. Biyik *et al.*, "Smart parking systems: Reviewing the literature, architecture and ways forward," *Smart Cities*, vol. 4, no. 2, 2021. [Online]. Available: https://www.mdpi.com/2624-6511/4/2/32

[4] National Payments Corporation of India, "NETC FASTag: National Electronic Toll Collection." [Online]. Available: https://www.npci.org.in/product/netc [Accessed: 22-Oct-2026].

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015, doi: 10.17487/RFC7519.

[6] Bluetooth Special Interest Group, "Bluetooth Core Specification: Generic Attribute Profile (GATT)." [Online]. Available: https://www.bluetooth.com/specifications/specs/core-specification/

[7] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: https://owasp.org/API-Security/

[8] Spring, "Spring Boot Reference Documentation." [Online]. Available: https://docs.spring.io/spring-boot/

[9] Spring, "Spring Security Reference." [Online]. Available: https://docs.spring.io/spring-security/reference/

[10] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." [Online]. Available: https://www.postgresql.org/docs/16/

[11] Expo, "Expo Documentation." [Online]. Available: https://docs.expo.dev/

[12] dotintent, "react-native-ble-plx: Bluetooth Low Energy library for React Native." [Online]. Available: https://github.com/dotintent/react-native-ble-plx

[13] Espressif Systems, "Arduino core for the ESP32." [Online]. Available: https://github.com/espressif/arduino-esp32

[14] B. Blanchon, “ArduinoJson: JSON library for Arduino and embedded C++.” [Online]. Available: https://arduinojson.org/

[15] M. Heaphy, “ESP32Servo: Arduino-compatible servo library for the ESP32.” [Online]. Available: https://github.com/madhephaestus/ESP32Servo

[16] Docker, “Docker Compose Documentation.” [Online]. Available: https://docs.docker.com/compose/

[17] Eclipse Foundation, “JaCoCo Java Code Coverage Library.” [Online]. Available: https://www.jacoco.org/jacoco/

[18] JUnit Team, “JUnit 5 User Guide.” [Online]. Available: https://junit.org/junit5/docs/current/user-guide/

[19] Mockito, “Mockito Framework Site.” [Online]. Available: https://site.mockito.org/

[20] J. Mandel, “JJWT: Java JWT.” [Online]. Available: https://github.com/jwtk/jjwt

# Appendix A

Links:

- GitHub: https://github.com/hari-hara-sudhann/dwara.git
- Demo Video: https://drive.google.com/file/d/1LZpc5eTHakTemG2XKVisyU7OeVnigFZF/view?usp=share_link